

议，所以我们将仅描述它最重要的机制，浏览某些内部的细节；这将使得我们能够通过树木看森林。每个洪流具有一个基础设施结点，称为追踪器（tracker）。当一个对等方加入某洪流时，它向追踪器注册自己，并周期性地通知追踪器它仍在该洪流中。以这种方式，追踪器跟踪正参与在洪流中的对等方。一个给定的洪流可能在任何时刻具有数以百计或数以千计的对等方。

如图 2-26 所示，当一个新的对等方 Alice 加入该洪流时，追踪器随机地从参与对等方的集合中选择对等方的一个子集（为了具体起见，设有 50 个对等方），并将这 50 个对等方的 IP 地址发送给 Alice。Alice 持有对等方的这张列表，试图与该列表上的所有对等方创建并行的 TCP 连接。我们称所有这样与 Alice 成功地创建一个 TCP 连接的对等方为“邻近对等方”（在图 2-26 中，Alice 显示了仅有三个邻近对等方。通常，她应当有更多的对等方）。随着时间的流逝，这些对等方中的某些可能离开，其他对等方（最初 50 个以外的）可能试图与 Alice 创建 TCP 连接。因此一个对等方的邻近对等方将随时间而波动。

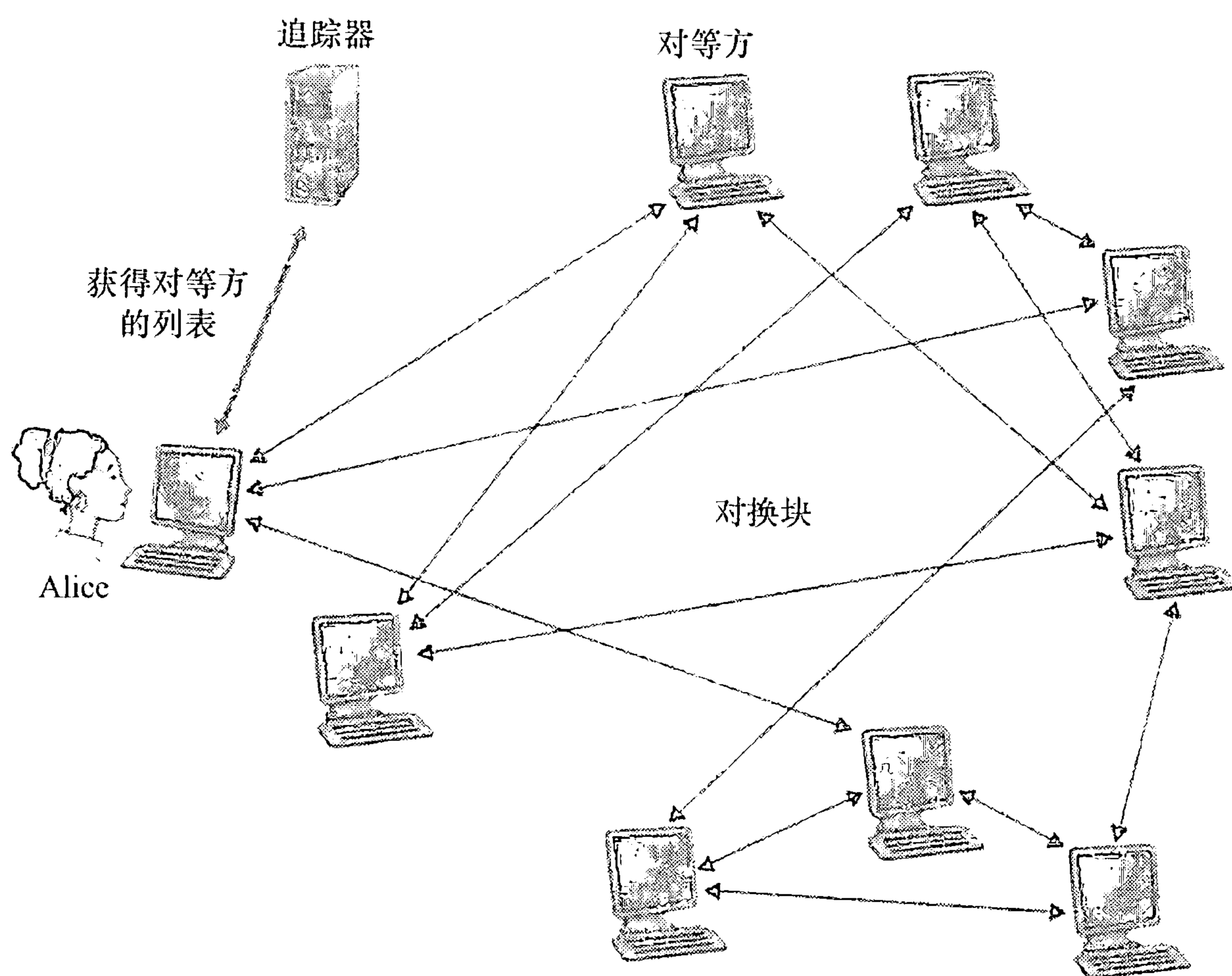


图 2-26 用 BitTorrent 分发文件

在任何给定的时间，每个对等方将具有来自该文件的块子集，并且不同的对等方具有不同的子集。Alice 周期性地（经 TCP 连接）询问每个邻近对等方它们所具有的块列表。如果 Alice 具有 L 个不同的邻居，她将获得 L 个块列表。有了这个信息，Alice 将对她当前还没有的块发出请求（仍通过 TCP 连接）。

因此在任何给定的时刻，Alice 将具有块的子集并知道它的邻居具有哪些块。利用这些信息，Alice 将作出两个重要决定。第一，她应当从她的邻居请求哪些块呢？第二，她应当向哪些向她请求块的邻居发送？在决定请求哪些块的过程中，Alice 使用一种称为最稀缺优先（rarest first）的技术。这种技术的思路是，针对她没有的块在她的邻居中决定最稀缺的块（最稀缺的块就是那些在她的邻居中副本数量最少的块），并首先请求那些最稀缺的块。这样，最稀缺块得到更为迅速的重新分发，其目标是（大致地）均衡每个块在洪流中的副本数量。

为了决定她响应哪个请求，BitTorrent 使用了一种机灵的对换算法。其基本想法是，Alice 根据当前能够以最高速率向她提供数据的邻居，给出其优先权。特别是，Alice 对于她的每个邻居都持续地测量接收到比特的速率，并确定以最高速率流入的 4 个邻居。每过 10 秒，她重新计算该速率并可能修改这 4 个对等方的集合。用 BitTorrent 术语来说，这 4 个对等方被称为疏通（unchoked）。重要的是，每过 30 秒，她也要随机地选择另外一个邻居并向其发送块。我们将这个被随机选择的对等方称为 Bob。因为 Alice 正在向 Bob 发送数据，她可能成为 Bob 前 4 位上载者之一，这样的话 Bob 将开始向 Alice 发送数据。如果 Bob 向 Alice 发送数据的速率足够高，Bob 接下来也能成为 Alice 的前 4 位上载者。换言之，每过 30 秒 Alice 将随机地选择一名新的对换伴侣并开始与那位伴侣进行对换。如果这两名对等方都满足此对换，它们将对方放入其前 4 位列表中并继续与对方进行对换，直到该对等方之一发现了一个更好的伴侣为止。这种效果是对等方能够以趋向于找到彼此的协调的速率上载。随机选择邻居也允许新的对等方得到块，因此它们能够具有对换的东西。除了这 5 个对等方（“前”4 个对等方和一个试探的对等方）的所有其他相邻对等方均被“阻塞”，即它们不能从 Alice 接收到任何块。BitTorrent 有一些有趣的机制没有在这里讨论，包括片（小块）、流水线、随机优先选择、残局模型和反怠慢 [Cohen 2003]。

刚刚描述的关于交换的激励机制经常被称为“一报还一报”（tit-for-tat）[Cohen 2003]。已证实这种激励方案能被回避 [Liogkas 2006; Locher 2006; Piatek 2007]。无论如何，BitTorrent “生态系统”取得了广泛成功，数以百万计的并发对等方在数十万条洪流中积极地共享文件。如果 BitTorrent 被设计为不采用一报还一报（或一种变种），然而在别的方面却完全相同的协议，BitTorrent 现在将很可能不复存在了，因为大多数用户将成为搭便车者了 [Sarouiu 2002]。

[Guo 2005; Piatek 2007] 提出了 BitTorrent 协议的有趣变种。此外，许多 P2P 直播流式应用如 PPLive 和 PPstream 从 BitTorrent 中获得了灵感 [Hei 2007]。

2.6.2 分布式散列表

在本节中，我们将考虑在 P2P 网络中怎样实现一个简单的数据库。我们先从描述这种简单数据库的集中式版本开始，该数据库只包含（键，值）对。例如：键可以是社会保险号，值可以是相应的人名；在这种情况下，键 - 值对的例子如（156-45-7081, Johnny Wu）。或者键可以是目录名（例如，电影、唱片和软件的名字），值可以是存储内容的 IP 地址；在这种情况下，键 - 值对的例子如（Led Zppelin IV, 128.187.123.38）。我们用键来查询数据库。如果在该数据库中有一个或多个键 - 值对匹配该查询键，该数据库就返回相应的值。例如，如果该数据库存储了社会保险号和它们对应的人名，我们能够用特定的社会保险号查询，该数据库就返回具有那个社会保险号的人的名字。或者，如果数据库存储了目录名及其相应的 IP 地址，我们能够用特定的目录名查询，该数据库返回存储该特定内容的 IP 地址。

构建这样一个数据库直接采用客户 - 服务器体系结构，以在一个中心服务器中存储所有（键，值）对。因此在本节中，我们将考虑构建这种数据库的一个分布式、P2P 的版本，在数以百万计的对等方上存储（键，值）对。在该 P2P 系统中，每个对等方将保持（键，值）对仅占总体的一个小子集。我们将允许任何对等方用一个特别的键来查询该分布式数据库。分布式数据库则将定位拥有该相应（键，值）对的对等方，然后向查询的对

等方返回该（键，值）对。任何对等方也将允许在数据库中插入新键 - 值对。这样一种分布式数据库被称为分布式散列表（Distributed Hash Table, DHT）。

在描述如何创建一个 DHT 的方法之前，我们首先描述在 P2P 文件共享环境中 DHT 服务的一个特定例子。在这种情况下，键是一个目录名，而值是具有该目录副本的对等方 IP 地址。因此，如果 Bob 和 Charlie 都具有一个最新 Linux 分发副本的话，则 DHT 数据库将包括下列两个键 - 值对：（Linux, IP_{Bob} ）和（Linux, $IP_{Charlie}$ ）。更为具体地说，因为 DHT 数据库分布在一些对等方上，所以某些对等方如 Dave 将对键“Linux”负责，并且将具有相应的键 - 值对。现在假设 Alice 要获得 Linux 的一个副本。显然，她在能够开始下载之前，先需要知道哪个对等方拥有 Linux 的副本。为此，她用“Linux”作为键来查询 DHT。该 DHT 则确定对等方 Dave 负责键“Linux”。DHT 则联系对等方 Dave，从 Dave 处获得键 - 值对（Linux, IP_{Bob} ）和（Linux, $IP_{Charlie}$ ），并将它们传送给 Alice。Alice 则能从 IP_{Bob} 或 $IP_{Charlie}$ 处下载最新 Linux 分发。

现在我们返回到为通用键 - 值对设计一个 DHT 这个一般性的问题上。构建 DHT 的一种幼稚的方法是跨越所有对等方随机地散布（键，值）对，让每个对等方维护一个所有参与对等方的列表。在该设计中，查询的对等方向所有其他对等方发送它的查询，并且包含与键匹配的（键，值）对的对等方能够用它们匹配的对进行响应。当然，这样的方法完全无扩展性，因为它将要求每个对等方不仅知道所有其他对等方（这样的对等方可能数以百万计！），而且更糟的是，要向所有对等方发送一个查询。

我们现在描述设计 DHT 的一种精确有效的方法。为此，我们现为每个对等方分配一个标识符，其中每个标识符是一个 $[0, 2^n - 1]$ 范围内的整数， n 取某些固定的值。值得注意的是这样的标识符能够由一个 n 比特表示法来表示。我们也要求每个键是同一范围内的一个整数。敏锐的读者也许已经观察到刚才描述的键的例子（社会保险号和目录名）并非整数。为在这些键范围外生成整数，我们将使用散列函数把每个键（如社会保险号）映射为 $[0, 2^n - 1]$ 范围内的一个整数。散列函数是一种多对一的函数，使两个不同的输入能够具有相同的输出（相同的整数），但是具有相同输出的似然性极低。（不熟悉散列函数的读者可以看一下第 7 章，其中更为细致地讨论了散列函数。）假定系统中的所有对等方可以使用散列函数。因此，当我们提及“键”，指的是初始键的散列值。因此，假如初始键是“Led Zppelin IV”，在 DHT 中使用的键将是等于“Led Zppelin IV”散列值的整数。如你猜测的那样，这就是“散列”被用于术语“分布式散列表”中的理由。

现在我们来考虑在 DHT 中存储（键，值）对的问题。此时的中心问题是定义为对等方分配键的规则。给定每个对等方具有一个整数标识符，每个键也是一个位于相同范围的整数，一种当然的方法是为其标识符最邻近该键的对等方分配一个（键，值）对。为实现这样的方案，我们将需要定义“最邻近”的含义。为了方便起见，我们将最邻近对等方定义为键的最邻近后继。为了加深理解，我们来看一个特定的例子。假设 $n = 4$ ，使所有对等方和键标识符都位于 $[0, 15]$ 范围内。进一步假定在该系统中有 8 个对等方，它们的标识符分别为 1、3、4、5、8、10、12 和 15。假定要在这 8 个对等方之一上存储（键，值）对（11, Johnny Wu）。但是放在哪个对等方上呢？使用我们定义的最邻近规则，因为对等方 12 是键 11 最邻近的后继，因此我们将（11, Johnny Wu）存储在对等方 12 上。[为了完成我们的“最邻近”定义，说明如下：如果该键恰好等于这些对等方标识符之一，我们在匹配的对等方中存储（键，值）对；并且如果该键大于所有对等方标识符，我们使用模

2ⁿ 规则，在具有最小标识符的对等方中存储（键，值）对。]

此时假定某对等方 Alice 要将一个（键，值）对插入 DHT。从概念上讲，这是直接的：她首先确定标识符最邻近该键的对等方；然后她向那个对等方发送一个报文，指示它存储该（键，值）对。但是 Alice 怎样确定最邻近该键的对等方呢？如果 Alice 要联系系统中所有对等方（对等方 ID 和相应的 IP 地址），她能够在本地确定最邻近对等方。但是这样的方法要求每个对等方联系在该 DHT 中的所有其他对等方，而这对于具有数以百万计对等方的大规模系统而言，是完全不现实的。

1. 环形 DHT

为了处理规模的问题，我们现在考虑将对等方组织为一个环。在这种环形设置中，每个对等方仅与它的直接后继和直接前任联系（模 2^n ）。在图 2-27a 中显示了这种环的一个例子。在该例中， n 仍取为 4，有与前例同样的 8 个对等方。每个对等方仅知道它的直接后继和直接前任，例如，对等方 5 知道对等方 8 和 4 的 IP 地址和标识符，但不必知道在该 DHT 中任何其他对等方的情况。对等方的这种环形设置是覆盖网络的一个特殊情况。在一个覆盖网络中，对等方形成了一个抽象的逻辑网，该网存在于由物理链路、路由器和主机组成的“底层”计算机网络之上。在覆盖网络中的链路不是物理链路，而仅是对等方对之间的虚拟联络。在图 2-27a 所示的覆盖网络中，有 8 个对等方和 8 条覆盖链路；在图 2-27b 所示的覆盖网络中，有 8 个对等方和 16 条覆盖链路。一条单一的覆盖链路通常使用了底层网络中的许多条物理链路和物理路由器。

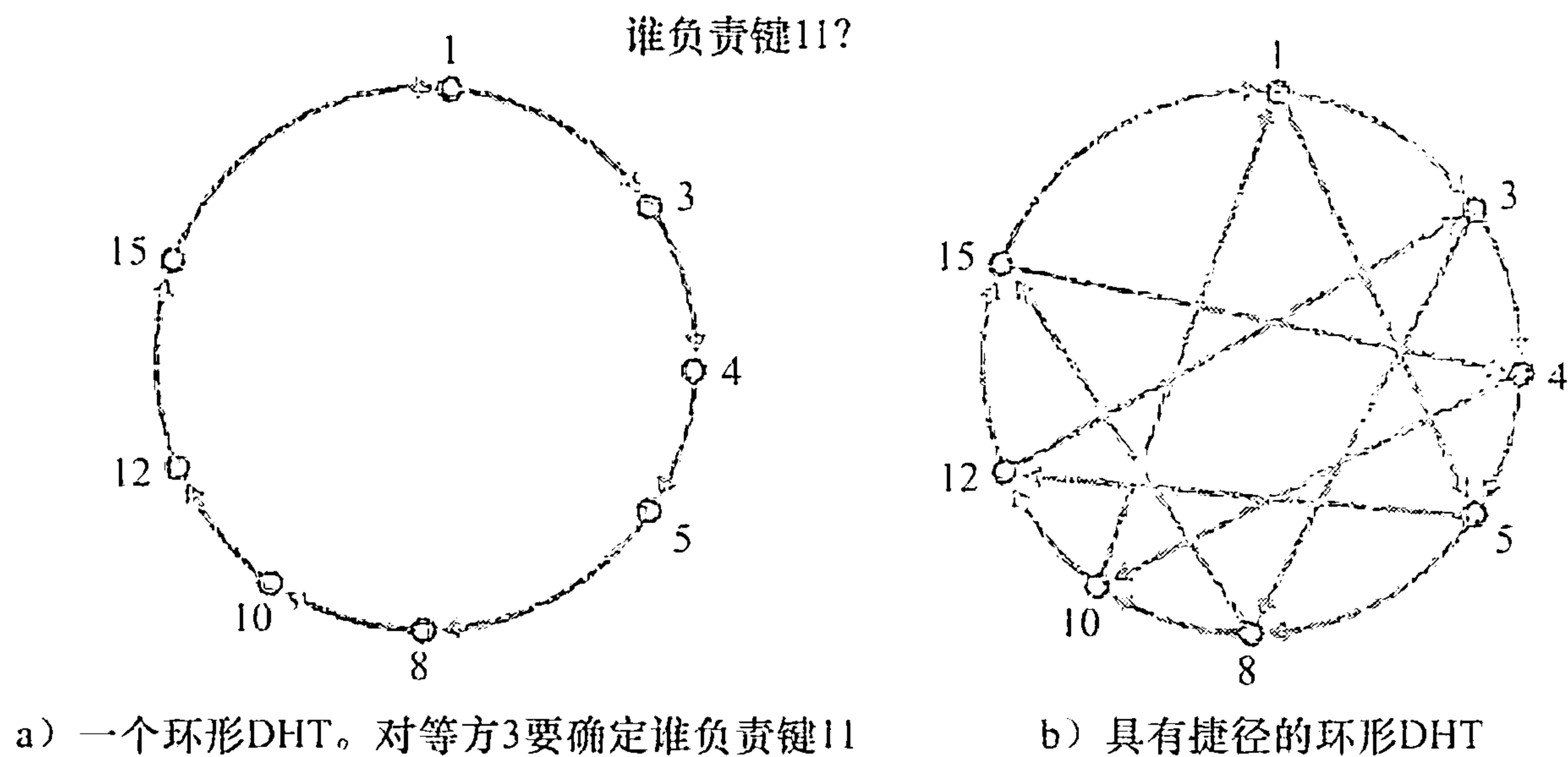


图 2-27 在环形 DHT 中确定最邻近键

使用图 2-27a 中的环形覆盖网络，现在假定对等方 3 要确定 DHT 中的哪个对等方负责键 11。使用该环形覆盖网络，初始对等方（对等方 3）生成一个报文，问“谁负责键 11？”并绕环顺时针发送该报文。无论何时某对等方接收到该报文，因为它知道其后继和前任的标识符，它能够确定是否由它负责（即最邻近）查询中的键。如果某对等方不负责该键，它只需将该报文发送给它的后继。因此，例如当对等方 4 接收到询问键 11 的报文，它确定自己不负责该键（因为其后继更邻近该键），故它只需将该报文传递给对等方 5。这个过程直到该报文到达对等方 12 才终止，对等方 12 确定自己是最邻近键 11 的对等方。此时，对等方 12 能够向查询的对等方即对等方 3 回送一个报文，指出自己负责键 11。

为减少每个对等方必须管理的覆盖信息的数量，环形 DHT 提供了一种非常精确有效

的解决方案。特别是，每个对等方只需要知道两个对等方，即它的直接后继和直接前任。但该解决方案也引入了一个新问题。尽管每个对等方仅知道两个邻居对等方，但为了找到负责的键（在最差的情况下），DHT 中的所有 N 个结点将必须绕环转发该报文；平均发送 $N/2$ 条报文。

所以，在设计 DHT 时，在每个对等方必须跟踪的邻居数量与 DHT 为解析一个查询而需要发送的报文数量之间存在着折中。一方面，如果每个对等方联系所有其他对等方（网状覆盖网络），则每个查询仅发送一个报文，但每个对等方必须关联 N 个对等方。在另一方面，使用环形 DHT，每个对等方仅知道两个对等方，但对每个查询平均要发送 $N/2$ 条报文。幸运的是，我们能够完善 DHT 设计，使每个对等方的邻居数量和每次查询的报文数量都保持在可接受的范围内。细化的方案之一是以该环形覆盖网络为基础，但增加“捷径”，使每个对等方不仅联系它的直接后继和直接前任，而且联系分布在环上的数量相对少的捷径对等方。这种具有某些捷径的环形 DHT 的例子显示在图 2-27b 中。使用捷径来加速查询报文的路由选择。具体来说，当某对等方接收到一条查询一个键的报文时，它向最接近该键的邻居（后继邻居或捷径邻居之一）转发该报文。所以，在图 2-27b 中，当对等方 4 接收到请求键 11 的报文，它确定（在它的邻居中）对该键最邻近的对等方是它的捷径邻居 10，并且直接向对等方 10 转发该报文。显然，捷径能够大大减少用于处理查询的报文数量。

下一个自然的问题是：“一个对等方应当有多少条捷径，哪些对等方应当成为这些捷径邻居？”该问题已经受到研究界的高度重视 [Balakrishnan 2003; Androutsellis – Theotokis 2004]。重要的是，研究表明 DHT 能被设计成每个对等方的邻居数量以及每个请求的报文数量均为 $O(\log N)$ ，其中 N 是对等方的数量。这种设计给出了使用网状和环形覆盖网络拓扑的两种极端解决方案之间的一种满意折中方案。

2. 对等方扰动

在 P2P 系统中，对等方能够不加警示地到来和离去。因此，当设计一个 DHT 时，我们也必须关注存在这种对等方扰动时维护 DHT 的情况。为了更好地理解处理这个问题所使用的方法，我们再次考虑图 2-27a 中的环形 DHT。为处理对等方扰动，我们此时将要求每个对等方联系其第一个和第二个后继（即知道它们的 IP 地址）；例如，对等方 4 此时联系对等方 5 和对等方 8。我们也要求每个对等方周期性地证实它的两个后继是存活的（例如，通过周期性地向它们发送 ping 报文并寻求响应）。现在我们考虑当某对等方突然离开时 DHT 如何维护 DHT。例如，假定图 2-27a 中的对等方 5 突然离开。在此情况下，因为对等方 5 不再响应 ping 报文，在离开的对等方之前的两个对等方（4 和 3）得知 5 已经离开。对等方 4 和 3 因此需要更新它们后继的状态信息。我们考虑对等方 4 更新其状态的方法：

- 对等方 4 用它的第二个后继（对等方 8）来代替它的第一个后继（对等方 5）。
- 然后对等方 4 向新的第一个后继询问它的直接后继（对等方 10）的标识符和 IP 地址。然后对等方 4 将对等方 10 标记为它的第二个后继。

在课后习题中，请你确定对等方 3 是如何更新它的覆盖路由选择信息的。

在简要地讨论了一个对等方离开时必须要做的事后，我们现在考虑当一个对等方要加入 DHT 时发生的事情。假如一个标识符为 13 的对等方要加入该 DHT，在加入时，它仅知

道对等方 1 存在于该 DHT 之中。对等方 13 将先要向对等方 1 发送一条报文，说“13 的前任和后继是什么？”该报文将通过 DHT 到达对等方 12，而它认识到自己将是 13 的前任，并且它的当前后继即对等方 15 将成为 13 的后继。接下来，对等方 12 向对等方 13 发送它的前任和后继信息。对等方 13 此时能够加入 DHT，标识它的后继为对等方 15，并通知对等方 12 它应当将其直接后继改为 13。

DHT 已经在实践中得到了广泛使用。例如，BitTorrent 使用 Kademlia DHT 来产生一个分布式跟踪器。在 BitTorrent 中，其键是洪流标识符而其值是当前参与洪流的所有对等方的 IP 地址 [Falkner 2007, Neglia 2007]。以这种方式，通过用某洪流标识符来查询 DHT，一个新到达的 BitTorrent 对等方能够确定负责该标识符（即在洪流中跟踪对等方）的对等方。在找到该对等方后，到达的对等方能够向它查询在洪流中的其他对等方列表。

2.7 TCP 套接字编程

我们已经看到了一些重要的网络应用，下面就探讨一下网络应用程序是如何实际编写的。在 2.1 节讲过，典型的网络应用是由一对程序（即客户程序和服务器程序）组成的，它们位于两个不同的端系统中。当运行这两个程序时，创建了一个客户进程和一个服务器进程，同时它们通过从套接字读出和写入数据彼此之间进行通信。开发者创建一个网络应用时，其主要任务就是编写客户程序和服务器程序的代码。

网络应用程序有两类。一类是实现在协议标准（如一个 RFC 或某种其他标准文档）中所定义的操作；这样的应用程序又称为“开放”的，因为定义其操作的这些规则人所共知。对于这样的实现，客户程序和服务器程序必须遵守由该 RFC 所规定的规则。例如，某客户程序可能是 FTP 协议客户端的一种实现，如在 2.3 节所描述，该协议由 RFC 959 明确定义；类似地，其服务器程序能够是 FTP 服务器协议的一种实现，也明确由 RFC 959 定义。如果一个开发者编写客户程序的代码，另一个开发者编写服务器程序的代码，并且两者都完全遵从该 RFC 的各种规则，那么这两个程序将能够交互操作。实际上，今天大多数网络应用程序涉及客户和服务器程序间的通信，这些程序都是由不同的程序员单独开发的。例如，与 Apache Web 服务器通信的 Firefox 浏览器，或与 BitTorrent 跟踪器通信的 BitTorrent 客户。

另一类网络应用程序是专用的网络应用程序。在这种情况下，由客户和服务器程序应用的应用层协议没有公开发布在某 RFC 中或其他地方。某单独的开发（或开发团队）创建了客户和服务器程序，并且该开发者用他的代码完全控制程序的功能。但是因为这些代码并没有实现一个开放的协议，其他独立的开发者将不能开发出和该应用程序交互的代码。

在本节中，我们将考察研发一个客户 - 服务器应用程序中的关键问题，我们将“亲历亲为”来实现一个非常简单的客户 - 服务器应用程序代码。在研发阶段，开发者必须最先做的一个决定是，应用程序是运行在 TCP 上还是运行在 UDP 上。前面讲过 TCP 是面向连接的，并且为两个端系统之间的数据流动提供可靠的字节流通道。UDP 是无连接的，从一个端系统向另一个端系统发送独立的数据分组，不对交付提供任何保证。前面也讲过当客户或服务器程序实现了一个由某 RFC 定义的协议，它应当使用与该协议关联的周知端口号；与之相反，当研发一个专用应用程序，研发者必须注意避免使用这样的周知端口号。

(端口号已在 2.1 节简要讨论过。它们将在第 3 章中更为详细地涉及。)

我们通过一个简单的 UDP 应用程序和一个简单的 TCP 应用程序来介绍 UDP 和 TCP 套接字编程。我们用 Python 语言来呈现这些简单的 TCP 和 UDP 程序。也可以用 Java、C 或 C++ 来编写这些程序，而我们选择用 Python 最主要原因是 Python 清楚地揭示了关键的套接字概念。使用 Python，代码的行数更少，并且向新编程人员解释每一行代码不会有太大困难。如果你不熟悉 Python，也用不着担心，只要你有过一些用 Java、C 或 C++ 编程的经验，就应该很容易看得懂下面的代码。

如果读者对用 Java 进行客户 - 服务器编程感兴趣，建议你去查看与本书配套的 Web 网站。事实上，能够在那里找到用 Java 编写的本节中的所有例子（和相关的实验）。如果读者对用 C 进行客户 - 服务器编程感兴趣，有一些优秀参考资料可供使用 [Donahoo 2001; Stevens 1997; Frost 1994; Kurose 1996]。我们下面的 Python 例子具有类似于 C 的外观和感觉。

2.7.1 UDP 套接字编程

在本小节中，我们将编写使用 UDP 的简单客户 - 服务器程序；在下一小节中，我们将编写使用 TCP 的简单程序。

2.1 节讲过，运行在不同机器上的进程彼此通过向套接字发送报文来进行通信。我们说过每个进程好比是一座房子，该进程的套接字则好比是一扇门。应用程序位于房子中门的一侧；运输层位于该门朝外的另一侧。应用程序开发者在套接字的应用层一侧可以控制所有东西；然而，它几乎无法控制运输层一侧。

现在我们仔细观察使用 UDP 套接字的两个通信进程之间的交互。在发送进程能够将数据分组推出套接字之门之前，当使用 UDP 时，必须先将目的地址附在该分组之上。在该分组传过发送方的套接字之后，因特网将使用该目的地址通过因特网为该分组选路到接收进程的套接字。当分组到达接收套接字时，接收进程将通过该套接字取回分组，进而检查分组的内容并采取适当的动作。

因此你可能现在想知道，附在分组上的目的地址包含了什么？如你所期待的，目的主机的 IP 地址是目的地址的一部分。通过在分组中包括目的地的 IP 地址，因特网中的路由器将能够通过因特网将分组选路到目的主机。但是因为一台主机可能运行许多网络应用进程，每个进程具有一个或多个套接字，所以在目的主机指定特定的套接字也是必要的。当生成一个套接字时，就为它分配一个称为端口号（port number）的标识符。因此，如你所期待的，分组的目的地址也包括该套接字的端口号。归纳起来，发送进程为分组附上的目的地址是由目的主机的 IP 地址和目的地套接字的端口号组成的。此外，如我们很快将看到的那样，发送方的源地址也是由源主机的 IP 地址和源套接字的端口号组成，该源地址也要附在分组之上。然而，将源地址附在分组之上通常并不是由 UDP 应用程序代码所为，而是由底层操作系统自动完成的。

我们将使用下列简单的客户 - 服务器应用程序来演示对于 UDP 和 TCP 的套接字编程：

- 1) 客户从其键盘读取一行字符并将数据向服务器发送。
- 2) 服务器接收该数据并将这些字符转换为大写。
- 3) 服务器将修改的数据发送给客户。
- 4) 客户接收修改的数据并在其监视器上将该行显示出来。

图 2-28 着重显示了客户和服务器的主要与套接字相关的活动，两者通过 UDP 运输服务进行通信。

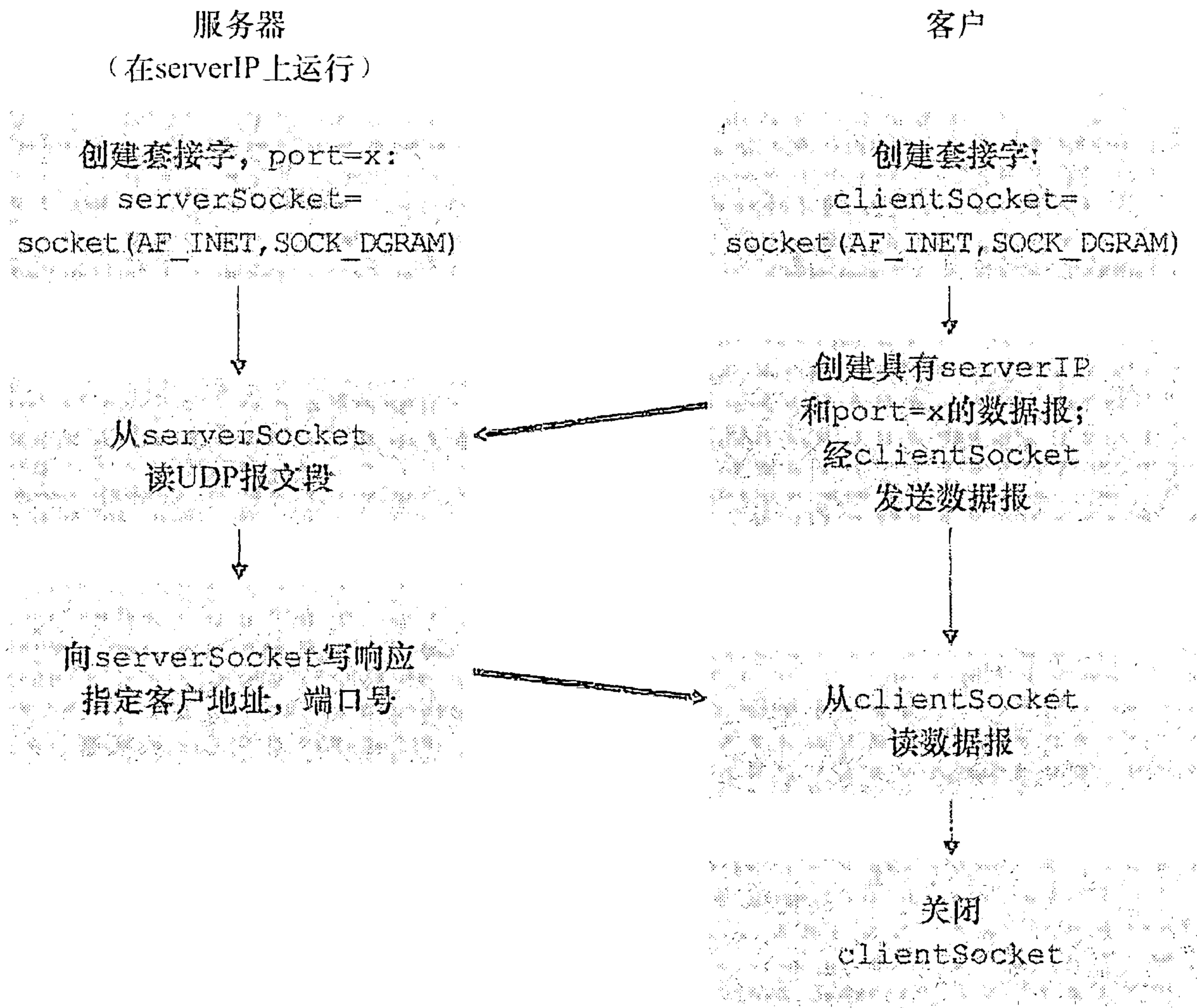


图 2-28 使用 UDP 的客户 - 服务器应用程序

现在我们自己动手来查看这对客户 - 服务器程序，用 UDP 实现这个简单应用程序。我们在每个程序后也提供一个详细、逐行的分析。我们将以 UDP 客户开始，该程序将向服务器发送一个简单的应用级报文。服务器为了能够接收并回答该客户的报文，它必须准备好并已经在运行，这就是说，在客户发送其报文之前，服务器必须作为一个进程正在运行。

客户程序被称为 UDPClient.py，服务器程序被称为 UDPServer.py。为了强调关键问题，我们有意提供最少的代码。“好代码”无疑将具有一些更为辅助性的代码行，特别是用于处理出现差错的情况。对于本应用程序，我们任意选择了 12000 作为服务器的端口号。

1. UDPClient.py

下面是该应用程序客户端的代码：

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)
message = raw_input('Input lowercase sentence:')
clientSocket.sendto(message,(serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print modifiedMessage
clientSocket.close()
```

现在我们在 UDPClient.py 中的各行代码。

```
from socket import *
```

该 socket 模块形成了在 Python 中所有网络通信的基础。包括了这行，我们将能够在程序中创建套接字。


```
serverName = 'hostname'
serverPort = 12000
```

第一行将变量 `serverName` 置为字符串“hostname”。这里，我们提供了或者包含服务器的 IP 地址（如“128.138.32.126”）或者包含服务器的主机名（如“cis.poly.edu”）的字符串。如果我们使用主机名，则将自动执行 DNS lookup 从而得到 IP 地址。第二行将整数变量 `serverPort` 置为 12000。

```
clientSocket = socket(socket.AF_INET, socket.SOCK_DGRAM)
```

该行创建了客户的套接字，称为 `clientSocket`。第一个参数指示了地址簇；特别是，`AF_INET` 指示了底层网络使用了 IPv4。（此时不必担心，我们将在第 4 章中讨论 IPv4。）第二个参数指示了该套接字是 `SOCK_DGRAM` 类型的，这意味着它是一个 UDP 套接字（而不是一个 TCP 套接字）。值得注意的是，当创建套接字时，我们并没有指定客户套接字的端口号；相反，我们让操作系统为我们做这件事。既然客户进程的门已经创建，我们将要生成通过该门发送的报文。

```
message = raw_input('Input lowercase sentence:')
```

`raw_input()` 是 Python 中的内置功能。当执行这条命令时，在客户上的用户将以单词“Input lowercase sentence:”进行提示，用户使用她的键盘来输入一行，这被放入变量 `message` 中。既然我们有了一个套接字和一条报文，我们将要通过该套接字向目的主机发送报文。

```
clientSocket.sendto(message, (serverName, serverPort))
```

在上述这行中，方法 `sendto()` 为报文附上目的地址（`serverName`，`serverPort`）并且向进程的套接字 `clientSocket` 发送结果分组。（如前面所述，源地址也附到分组上，尽管这是自动完成的，而不是显式地由代码完成的。）经一个 UDP 套接字发送一个客户到服务器的报文非常简单！在发送分组之后，客户等待接收来自服务器的数据。

```
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
```

对于上述这行，当一个来自因特网的分组到达该客户套接字时，该分组的数据被放置到变量 `modifiedMessage` 中，其源地址被放置到变量 `serverAddress` 中。变量 `serverAddress` 包含了服务器的 IP 地址和服务器的端口号。程序 `UDPClient` 实际上并不需要服务器的地址信息，因为它从起始就已经知道了该服务器地址；而这行 Python 代码仍然提供了服务器的地址。方法 `recvfrom` 也取缓存长度 2048 作为输入。（该缓存长度用于多种目的。）

```
print modifiedMessage
```

这行在用户显示器上打印出 `modifiedMessage`。它应当是变用户键入的原始行，现在只是变为大写的了。

```
clientSocket.close()
```

该行关闭了套接字。然后关闭了该进程。

2. UDPServer.py

现在来看看这个应用程序的服务器端：

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print "The server is ready to receive"
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.upper()
    serverSocket.sendto(modifiedMessage, clientAddress)
```


注意到 UDPServer 的开始部分与 UDPClient 类似。它也是导入套接字模块，也将整数变量 serverPort 设置为 12000，并且也创建套接字类型 SOCK_DGRAM（一种 UDP 套接字）。与 UDPClient 有很大不同的第一行代码是：

```
serverSocket.bind('', serverPort)
```

上面行将端口号 12000 与个服务器的套接字绑定（即分配）在一起。因此在 UDPServer 中，（由应用程序开发者编写的）代码显式地为该套接字分配一个端口号。以这种方式，当任何人向位于该服务器的 IP 地址的端口 12000 发送一个分组，该分组将指向该套接字。UDPServer 然后进入一个 while 循环；该 while 循环将允许 UDPServer 无限期地接收并处理来自客户的分组。在该 while 循环中，UDPServer 等待一个分组的到达。

```
message, clientAddress = serverSocket.recvfrom(2048)
```

这行代码类似于我们在 UDPClient 中看到的。当某分组到达该服务器的套接字时，该分组的数据被放置到变量 message 中，其源地址被放置到变量 clientAddress 中。变量 clientAddress 包含了客户的 IP 地址和客户的端口号。这里，UDPServer 将利用该地址信息，因为它提供了返回地址，类似于普通邮政邮件的返回地址。使用该源地址信息，服务器此时知道了它应当将回答发向何处。

```
modifiedMessage = message.upper()
```

此行是这个简单应用程序的关键部分。它获取由客户发送的行并使用方法 upper() 将其转换为大写。

```
serverSocket.sendto(modifiedMessage, clientAddress)
```

最后一行将该客户的地址（IP 地址和端口号）附到大写报文上，并将所得的分组发送到服务器的套接字中。（如前面所述，服务器地址也附在分组上，尽管这是自动而不是显式地由代码完成的。）因特网则将分组交付到该客户地址。在服务器发送该分组后，它仍维持在 while 循环中，等待（从运行在任一台主机上的任何客户发送的）另一个 UDP 分组到达。

为了测试这对程序，可在一台主机上运行 UDPClient.py，并在另一台主机上运行 UDPServer.py。保证在 UDPClient.py 中包括适当的服务器主机名或 IP 地址。接下来，在服务器主机上执行编译的服务器程序 UDPServer.py。这在服务器上创建了一个进程，等待着某个客户与之联系。然后，在客户主机上执行编译的客户器程序 UDPClient.py。这在客户上创建了一个进程。最后，在客户上使用应用程序，键入一个句子并以回车结束。

可以通过稍加修改上述客户和服务器程序来研制自己的 UDP 客户 - 服务器程序。例如，不必将所有字母转换为大写，服务器可以计算字母 s 出现的次数并返回该数字。或者能够修改客户程序，使得收到一个大写的句子后，用户能够向服务器继续发送更多的句子。

2.7.2 TCP 套接字编程

与 UDP 不同，TCP 是一个面向连接的协议。这意味着在客户和服务器能够开始互相发送数据之前，它们先要握手和创建一个 TCP 连接。TCP 连接的一端与客户套接字相联系，另一端与服务器套接字相联系。当创建该 TCP 连接时，我们将其与客户套接字地址（IP 地址和端口号）和服务器套接字地址（IP 地址和端口号）关联起来。使用创建的 TCP 连接，当一侧要向另一侧发送数据时，它只需经过其套接字将数据丢给 TCP 连接。这与

UDP 不同，UDP 服务器在将分组丢进套接字之前必须为其附上一个目的地地址。

现在我们仔细观察一下 TCP 中客户程序和服务器程序的交互。客户具有向服务器发起接触的任务。服务器为了能够对客户的初始接触做出反应，服务器必须已经准备好。这意味着两件事。第一，与在 UDP 中的情况一样，TCP 服务器在客户试图发起接触前必须作为进程运行起来。第二，服务器程序必须具有一扇特殊的门，更精确地说是一个特殊的套接字，该门欢迎来自运行在任意主机上的客户进程的某些初始接触。使用房子/门来比喻进程/套接字，有时我们将客户的初始接触称为“敲欢迎之门”。

随着服务器进程的运行，客户进程能够向服务器发起一个 TCP 连接。这是由客户程序通过创建一个 TCP 套接字完成的。当该客户生成其 TCP 套接字时，它指定了服务器中的欢迎套接字的地址，即服务器主机的 IP 地址及其套接字的端口号。生成其套接字后，该客户发起了一个三次握手并创建与服务器的一个 TCP 连接。发生在运输层的三次握手，对于客户和服务器程序是完全透明的。

在三次握手期间，客户进程敲服务器进程的欢迎之门。当该服务器“听”到敲门时，它将生成一扇新门（更精确地讲是一个新套接字），它专门用于特定的客户。在我们下面的例子中，欢迎之门是一个我们称为 `serverSocket` 的 TCP 套接字对象；它专门对客户进行连接的新生成的套接字，称为连接套接字（`connection Socket`）。初次遇到 TCP 套接字的学生有时会混淆欢迎套接字（这是所有要与服务器通信的客户的起始接触点）和每个新生成的服务器侧的连接套接字（这是随后为与每个客户通信而生成的套接字）。

从应用程序的观点来看，客户套接字和服务器连接套接字直接通过一根管道连接。如图 2-29 所示，客户进程可以向它的套接字发送任意字节，并且 TCP 保证服务器进程能够按发送的顺序接收（通过连接套接字）每个字节。TCP 因此在客户和服务器进程之间提供了可靠服务。此外，就像人们可以从同一扇门进和出一样，客户进程不仅能向它的套接字发送字节，也能从中接收字节；类似地，服务器进程不仅从它的连接套接字接收字节，也能向其发送字节。

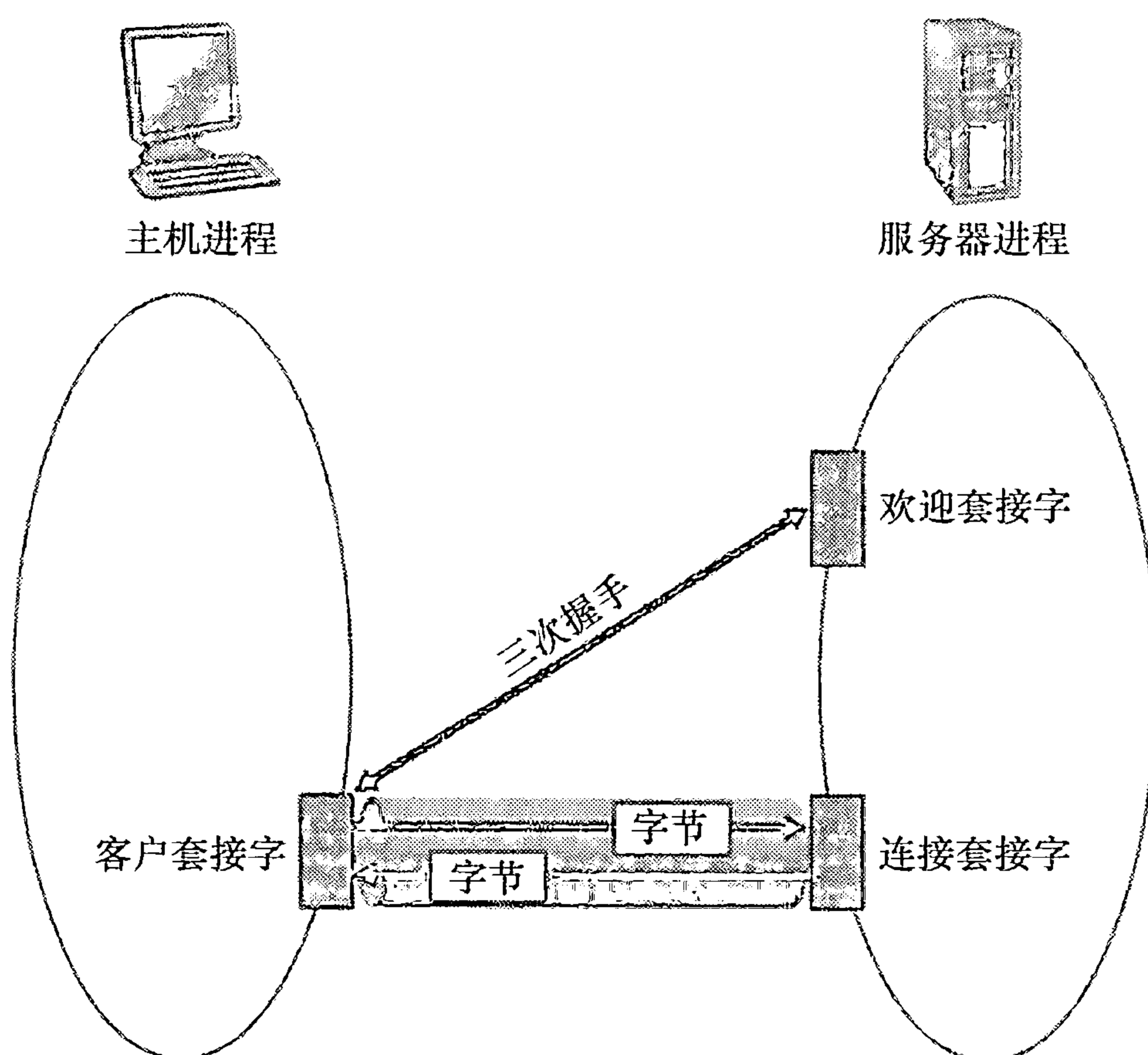


图 2-29 TCPServer 进程有两个套接字

我们使用同样简单的客户 - 服务器应用程序来展示 TCP 套接字编程：客户向服务器发送一行数据，服务器将这行改为大写并回送给客户。图 2-30 着重显示了客户和服务器的主要与套接字相关的活动，两者通过 TCP 运输服务进行通信。

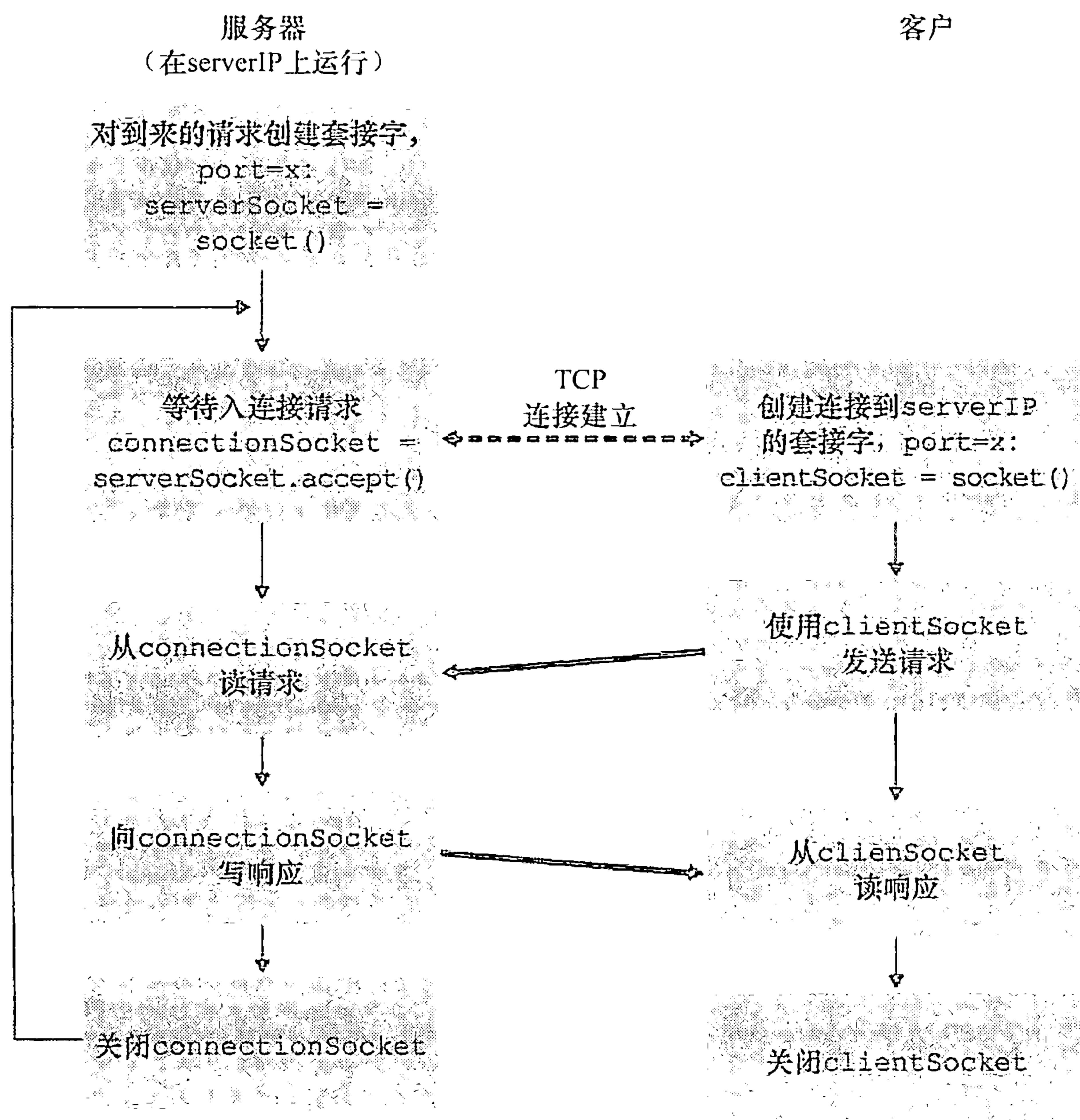


图 2-30 使用 TCP 的客户 - 服务器应用程序

1. TCPClient.py

这里给出了应用程序客户端的代码：

```

from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = raw_input('Input lowercase sentence:')
clientSocket.send(sentence)
modifiedSentence = clientSocket.recv(1024)
print 'From Server:', modifiedSentence
clientSocket.close()
  
```

现在我们查看这些代码中的与 UDP 实现有很大差别的各行。首当其冲的行是客户套接字的创建。

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

该行创建了客户的套接字，称为 clientSocket。第一个参数仍指示底层网络使用的是 IPv4。第二个参数指示该套接字是 SOCK_STREAM 类型。这表明它是一个 TCP 套接字（而不是一个 UDP 套接字）。值得注意的是当我们创建该客户套接字时仍未指定其端口号；相反，我们让操作系统为我们做此事。此时的下一行代码与我们在 UDPClient 中看到的极为不同：


```
clientSocket.connect((serverName,serverPort))
```

前面讲过在客户能够使用一个 TCP 套接字向服务器发送数据之前（反之亦然），必须在客户与服务器之间创建一个 TCP 连接。上面这行就发起了客户和服务器的这条 TCP 连接。connect() 方法的参数是这条连接中服务器端的地址。这行代码执行完后，执行三次握手，并在客户和服务器之间创建起一条 TCP 连接。

```
sentence = raw_input('Input lowercase sentence:')
```

如同 UDPClient 一样，上一行从用户获得了一个句子。字符串 sentence 连续收集字符直到用户键入回车以终止该行为止。代码的下一行也与 UDPClient 极为不同：

```
clientSocket.send(sentence)
```

上一行通过该客户的套接字并进入 TCP 连接发送字符串 sentence。值得注意的是，该程序并未显式地创建一个分组并为该分组附上目的地址，而使用 UDP 套接字却要那样做。相反，客户程序只是将字符串 sentence 中的字节放入该 TCP 连接中去。客户然后就等待接收来自服务器的字节。

```
modifiedSentence = clientSocket.recv(2048)
```

当字符到达服务器时，它们被放置在字符串 modifiedSentence 中。字符继续积累在 modifiedSentence 中，直到收到回车符才会结束该行。在打印大写句子后，我们关闭客户的套接字。

```
clientSocket.close()
```

最后一行关闭了套接字，因此关闭了客户和服务器的 TCP 连接。它引起客户中的 TCP 向服务器中的 TCP 发送一条 TCP 报文（参见 3.5 节）。

2. TCPServer.py

现在我们先看一下服务器程序。

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print 'The server is ready to receive'
while 1:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024)
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence)
    connectionSocket.close()
```

现在我们来看看上述与 UDPServer 及 TCPClient 有显著不同的代码行。与 TCPClient 相同的是，服务器创建一个 TCP 套接字，执行：

```
serverSocket=socket(AF_INET,SOCK_STREAM)
```

与 UDPServer 类似，我们将服务器的端口号 serverPort 与该套接字关联起来：

```
serverSocket.bind(('',serverPort))
```

但对 TCP 而言，serverSocket 将是我们的欢迎套接字。在创建这扇欢迎之门后，我们将等待并聆听某个客户敲门：

```
serverSocket.listen(1)
```

该行让服务器聆听来自客户的 TCP 连接请求。其中参数定义了请求连接的最大数（至少为 1）。

```
connectionSocket, addr = serverSocket.accept()
```


当客户敲该门时，程序为 `serverSocket` 调用 `accept()`，这在服务器中创建了一个称为 `connectionSocket` 的新套接字，由这个特定的客户专用。客户和服务器则完成了握手，在客户的 `clientSocket` 和服务器的 `serverSocket` 之间创建了一个 TCP 连接。借助于创建的 TCP 连接，客户与服务器现在能够通过该连接相互发送字节。使用 TCP，从一侧发送的所有字节不仅确保到达另一侧，而且确保按序到达。

```
connectionSocket.close()
```

在此程序中，在向客户发送修改的句子后，我们关闭了该连接套接字。但由于 `serverSocket` 保持打开，所以另一个客户此时能够敲门并向该服务器发送一个句子要求修改。

我们现在完成了 TCP 套接字编程的讨论。建议你在两台单独的主机上运行这两个程序，也可以修改它们以达到稍微不同的目的。你应当将前面两个 UDP 程序与这两个 TCP 程序进行比较，观察它们的不同之处。你也应当做在第 2、4 和 7 章后面描述的套接字编程作业。最后，我们希望在掌握了这些和更先进的套接字程序后的某天，你将能够编写你自己的流行网络应用程序，变得非常富有和声名卓著，并记得本书的作者！

2.8 小结

在本章中，我们从概念和实现两方面对网络应用进行了学习。我们学习了被因特网应用普遍采用的客户 - 服务器模式，并且知道了该模式在 HTTP、FTP、SMTP、POP3 和 DNS 等协议中的使用。我们已经更为详细地学习了这些重要的应用层协议以及与之对应的相关应用（Web、文件传输、电子邮件和 DNS）。我们也已学习了日益流行的 P2P 体系结构以及它如何应用在许多应用程序中。我们还探讨了使用套接字 API 构建网络应用程序的方法。我们考察了面向连接的（TCP）和无连接的（UDP）端到端传输服务中的套接字应用。至此，我们在分层的网络体系结构中的向下之旅已经完成了第一步。

在本书一开始的 1.1 节中，我们对协议给出了一个相当含糊的框架性定义：“在两个或多个通信实体之间交换报文的格式和次序，以及对某报文或其他事件传输和/或接收所采取的动作。”本章中的内容，特别是我们对 HTTP、FTP、SMTP、POP3 和 DNS 协议进行的细致研究，已经为这个定义加入了相当可观的实质性的内容。协议是网络连接中的核心概念；对应用层协议的学习，为我们提供了有关协议内涵的更为直觉的认识。

在 2.1 节中，我们描述了 TCP 和 UDP 为调用它们的应用提供的服务模型。当我们在 2.7 节中开发运行在 TCP 和 UDP 之上的简单应用程序时，我们对这些服务模型进行了更加深入的观察。然而，我们几乎没有介绍 TCP 和 UDP 是如何提供这种服务模型的。例如，我们知道 TCP 提供了一种可靠数据服务，但我们未说它是如何做到这一点的。在下一章中我们将不仅关注运输协议是什么，而且还关注它如何工作以及为什么要这么做。

有了因特网应用程序结构和应用层协议的知识之后，我们现在准备继续沿该协议栈向下，在第 3 章中探讨运输层。

课后习题和问题



复习题

2.1 节

R1. 列出 5 种非专用的因特网应用及它们所使用的应用层协议。

- R2. 网络体系结构与应用程序体系结构之间有什么区别？
- R3. 对两进程之间的通信会话而言，哪个进程是客户，哪个进程是服务器？
- R4. 对一个 P2P 文件共享应用，你同意“一个通信会话不存在客户端和服务端的概念”的说法吗？为什么？
- R5. 运行在一台主机上的一个进程，使用什么信息来标识运行在另一台主机上的进程？
- R6. 假定你想尽快地处理从远程客户到服务器的事务，你将使用 UDP 还是 TCP？为什么？
- R7. 参见图 2-4，我们看到在该图所列出的应用程序没有一个同时既要求无数据丢失又要求定时的。你能设想一个既要求无数据丢失又高度时间敏感的应用程序吗？
- R8. 列出一个运输协议能够提供的 4 种宽泛类型的服务。对于每种服务类型，指出是 UDP 还是 TCP（或这两种协议）提供这样的服务？
- R9. 前面讲过 TCP 能用 SSL 来强化，以提供进程到进程的安全性服务，包括加密。SSL 运行在运输层还是应用层？如果某应用程序研制者想要用 SSL 来强化 UDP，该研制者应当做些什么工作？

2.2 ~ 2.5 节

- R10. 握手协议的作用是什么？
- R11. 为什么 HTTP、FTP、SMTP 及 POP3 都运行在 TCP，而不是 UDP 上？
- R12. 考虑一个电子商务网站需要保留每一个客户的购买记录。描述如何使用 cookie 来完成该功能？
- R13. 描述 Web 缓存器是如何减少接收被请求的对象的时延的。Web 缓存器将减少一个用户请求的所有对象或只是其中的某些对象的时延吗？为什么？
- R14. Telnet 到一台 Web 服务器并发送一个多行的请求报文。在该请求报文中包含 If - modified - since：首部行，迫使响应报文中出现“304 Not Modified”状态代码。
- R15. 为什么说 FTP 在“带外”发送控制信息？
- R16. 假定 Alice 使用一个基于 Web 的电子邮件账户（例如 Hotmail 或 gmail）向 Bob 发报文，而 Bob 使用 POP3 从他的邮件服务器访问自己的邮件。讨论是怎样从 Alice 主机到 Bob 主机得到该报文的。要列出在两台主机间移动该报文时所使用的各种应用层协议。
- R17. 将你最近收到的报文首部打印出来。其中有多少 Received：首部行？分析该报文的首部行中的每一行。
- R18. 从用户的观点看，POP3 协议中下载并删除模式和下载并保留模式有什么区别吗？
- R19. 一个机构的 Web 服务器和邮件服务器可以有完全相同的主机名别名（例如，foo.com）吗？包含邮件服务器主机名的 RR 有什么样的类型？
- R20. 仔细检查收到的电子邮件，查找由使用 .edu 电子邮件地址的用户发送的报文首部。从其首部，能够确定发送该报文的主机的 IP 地址吗？对于由 gmail 账号发送的报文做相同的事。

2.6 节

- R21. 在 BitTorrent 中，假定 Alice 向 Bob 提供一个 30 秒间隔的文件块吞吐量。Bob 将必须进行回报，在相同的间隔中向 Alice 提供文件块吗？为什么？
- R22. 考虑一个新对等方 Alice 加入 BitTorrent 而不拥有任何文件块。没有任何块，因此她没有任何东西可上载，她无法成为任何其他对等方的前 4 位上载者。那么 Alice 将怎样得到她的第一个文件块呢？
- R23. 覆盖网络是什么？它包括路由器吗？在覆盖网络中边是什么？
- R24. 考虑一个具有网状覆盖网络拓扑的 DHT（即每个对等方跟踪系统中的所有对等方）。这样设计的优点和缺点各是什么？环形 DHT（无捷径）的优点和缺点各是什么？
- R25. 列出至少 4 个不同的应用，它们本质上适合 P2P 体系结构。（提示：文件分布和即时讯息是两个这样的应用。）

2.7 节

- R26. 2.7 节中所描述的 UDP 服务器仅需要一个套接字，而 TCP 服务器需要两个套接字。为什么？如果

TCP 服务器支持 n 个并行连接，每条连接来自不同的客户主机，那么 TCP 服务器将需要多少个套接字？

- R27. 对于 2.7 节所描述的运行在 TCP 之上的客户 - 服务器应用程序，服务器程序为什么必须先于客户程序运行？对于运行在 UDP 之上的客户 - 服务器应用程序，客户程序为什么可以先于服务器程序运行？



习题

P1. 是非判断题。

- 假设用户请求由某些文本和 3 幅图像组成的 Web 页面。对于这个页面，客户将发送一个请求报文并接收 4 个响应报文。
- 两个不同的 Web 页面（例如，www.mit.edu/research.html 及 www.mit.edu/students.html）可以通过同一个持续连接发送。
- 在浏览器和初始服务器之间使用非持续连接的话，一个 TCP 报文段是可能携带两个不同的 HTTP 服务请求报文的。
- 在 HTTP 响应报文中的 Date：首部指出了该响应中对象最后一次修改的时间。
- HTTP 响应报文决不会具有空的报文体。

P2. 阅读有关 FTP 的 RFC 959。列出这个 RFC 所支持的所有客户命令。

P3. 考虑一个 HTTP 客户要获取一个给定 URL 的 Web 页面。该 HTTP 服务器的 IP 地址开始时并不知道。在这种情况下，除了 HTTP 外，还需要什么运输层和应用层协议？

P4. 考虑当浏览器发送一个 HTTP GET 报文时，通过 Wireshark 俘获到下列 ASCII 字符串（即这是一个 HTTP GET 报文的实际内容）。字符 `<cr>` `<lf>` 是回车和换行符（即下面文本中的斜体字符串 `<cr>` 表示了单个回车符，该回车符包含在 HTTP 首部中的相应位置）。回答下列问题，指出你在下面 HTTP GET 报文中找到答案的地方。

```
GET /cs453/index.html HTTP/1.1<cr><lf>Host: gai
a.cs.umass.edu<cr><lf>User-Agent: Mozilla/5.0 (
Windows;U; Windows NT 5.1; en-US; rv:1.7.2) Gec
ko/20040804 Netscape/7.2 (ax) <cr><lf>Accept:ex
t/xml, application/xml, application/xhtml+xml, text
/html;q=0.9, text/plain;q=0.8,image/png,*/*;q=0.5
<cr><lf>Accept-Language: en-us,en;q=0.5<cr><lf>Accept-
Encoding: zip,deflate<cr><lf>Accept-Charset: ISO
-8859-1,utf-8;q=0.7,*;q=0.7<cr><lf>Keep-Alive: 300<cr>
<lf>Connection:keep-alive<cr><lf><cr><lf>
```

- 由浏览器请求的文档的 URL 是什么？
- 该浏览器运行的是 HTTP 的何种版本？
- 该浏览器请求的是一条非持续连接还是一条持续连接？
- 该浏览器所运行的主机的 IP 地址是什么？
- 发起该报文的浏览器的类型是什么？在一个 HTTP 请求报文中，为什么需要浏览器类型？

P5. 下面文本中显示的是来自服务器的回答，以响应上述问题中 HTTP GET 报文。回答下列问题，指出你在下面报文中找到答案的地方。

```
HTTP/1.1 200 OK<cr><lf>Date: Tue, 07 Mar 2008
12:39:45GMT<cr><lf>Server: Apache/2.0.52 (Fedora)
<cr><lf>Last-Modified: Sat, 10 Dec2005 18:27:46
GMT<cr><lf>ETag: "526c3-f22-a88a4c80"<cr><lf>Accept-
Ranges: bytes<cr><lf>Content-Length: 3874<cr><lf>
Keep-Alive: timeout=max=100<cr><lf>Connection:
Keep-Alive<cr><lf>Content-Type: text/html; charset=
ISO-8859-1<cr><lf><cr><lf><!doctype html public "-
```



```
//w3c//dtd html 4.0 transitional//en"><lf><html><lf>
<head><lf> <meta http-equiv="Content-Type"
content="text/html; charset=iso-8859-1"><lf> <meta
name="GENERATOR" content="Mozilla/4.79 [en] (Windows NT
5.0; U) Netscape]"><lf> <title>CMPSCI 453 / 591 /
NTU-ST550A Spring 2005 homepage</title><lf></head><lf>
<much more document text following here (not shown)>
```

- a. 服务器能否成功地找到那个文档？该文档提供回答是什么时间？
 - b. 该文档最后修改是什么时间？
 - c. 文档中被返回的字节有多少？
 - d. 文档被返回的前 5 个字节是什么？该服务器同意一条持续连接吗？
- P6. 获取 HTTP/1.1 规范（RFC 2616）。回答下面问题：
- a. 解释在客户和服务器之间用于指示关闭持续连接的信令机制。客户、服务器或两者都能发送信令通知连接关闭了吗？
 - b. HTTP 提供了什么加密服务？
 - c. 一个客户能够与一个给定的服务器打开 3 条或更多条并发连接吗？
 - d. 如果一个服务器或一个客户检测到连接已经空闲一段时间，该服务器或客户可以关闭两者之间的传输连接。一侧开始关闭连接而另一侧通过该连接传输数据是可能的吗？请解释。
- P7. 假定你在浏览器中点击一条超链接获得 Web 页面。相关联的 URL 的 IP 地址没有缓存在本地主机上，因此必须使用 DNS lookup 以获得该 IP 地址。如果主机从 DNS 得到 IP 地址之前已经访问了 n 个 DNS 服务器；相继产生的 RTT 依次为 $RTT_1, \dots, RTT_n$ 。进一步假定与链路相关的 Web 页面只包含一个对象，即由少量的 HTML 文本组成。令 RTT_0 表示本地主机和包含对象的服务器之间的 RTT 值。假定该对象传输时间为零，则从客户点击该超链接到它接收到该对象需要多长时间？
- P8. 参照习题 P7，假定在同一服务器上某 HTML 文件引用了 8 个非常小的对象。忽略发送时间，在下列情况下需要多长时间：
- a. 没有并行 TCP 连接的非持续 HTTP。
 - b. 配置有 5 个并行连接的非持续 HTTP。
 - c. 持续 HTTP。
- P9. 考虑图 2-12，其中有一个机构的网络和因特网相连。假定对象的平均长度为 850 000 比特，从这个机构网的浏览器到初始服务器的平均请求率是每秒 16 个请求。还假定从接入链路的因特网一侧的路由器转发一个 HTTP 请求开始，到接收到其响应的平均时间是 3 秒（参见 2.2.5 节）。将总的平均响应时间建模为平均接入时延（即从因特网路由器到机构路由器的时延）和平均因特网时延之和。对于平均接入时延，使用 $\Delta/(1 - \Delta\beta)$ ，式中 Δ 是跨越接入链路发送一个对象的平均时间， β 是对象对该接入链路的平均到达率。
- a. 求出总的平均响应时间。
 - b. 现在假定在这个机构 LAN 中安装了一个缓存器。假定命中率为 0.4，求出总的响应时间。
- P10. 考虑一条 10 米短链路，某发送方经过它能够以 150bps 速率双向传输。假定包含数据的分组是 100 000 比特长，仅包含控制（如 ACK 或握手）的分组是 200 比特长。假定 N 个并行连接每个都获得 $1/N$ 的链路带宽。现在考虑 HTTP 协议，并且假定每个下载对象是 100Kb 长，这些初始下载对象包含 10 个来自相同发送方的引用对象。在这种情况下，经非持续 HTTP 的并行实例的并行下载有意义吗？现在考虑持续 HTTP。你期待这比非持续的情况有很大增益吗？评价并解释你的答案。
- P11. 考虑在前一个习题中引出的情况。现在假定该链路由 Bob 和 4 个其他用户所共享。Bob 使用非持续 HTTP 的并行实例，而其他 4 个用户使用无并行下载的非持续 HTTP。
- a. Bob 的并行连接能够帮助他更快地得到 Web 页面吗？
 - b. 如果所有 5 个用户打开 5 个非持续 HTTP 并行实例，那么 Bob 的并行连接仍将是有益的吗？为什么？

- P12. 写一个简单的 TCP 程序，使服务器接收来自客户的行并将其打印在服务器的标准输出上。（可以通过修改本书中的 TCPServer.py 程序实现上述任务。）编译并执行你的程序。在另一台有浏览器的机器上，设置浏览器的代理服务器为你正在运行服务器程序的机器，同时适当地配置端口号。这时你的浏览器向服务器发送 GET 请求报文，你的服务器应当在其标准输出上显示该报文。使用这个平台来确定你的浏览器是否对本地缓存的对象产生了条件 GET 报文。
- P13. SMTP 中的 MAIL FROM 与该邮件报文自身中的 From: 之间有什么不同？
- P14. SMTP 是怎样标识一个报文体结束的？HTTP 是怎样做的呢？HTTP 能够使用与 SMTP 标识一个报文体结束相同的方法吗？试解释。
- P15. 阅读用于 SMTP 的 RFC 5321。MTA 代表什么？考虑下面收到的垃圾邮件（从一份真实垃圾邮件修改得到）。假定这封垃圾邮件的唯一始作俑者是 malicious，而其他主机是诚实的，指出是该 malicious 主机产生了这封垃圾邮件。

```

From - Fri Nov 07 13:41:30 2008
Return-Path: <tennis5@pp33head.com>
Received: from barmail.cs.umass.edu
(barmail.cs.umass.edu [128.119.240.3]) by cs.umass.edu
(8.13.1/8.12.6) for <hg@cs.umass.edu>; Fri, 7 Nov 2008
13:27:10 -0500
Received: from asusus-4b96 (localhost [127.0.0.1]) by
barmail.cs.umass.edu (Spam Firewall) for
<hg@cs.umass.edu>; Fri, 7 Nov 2008 13:27:07 -0500
(EST)
Received: from asusus-4b96 ([58.88.21.177]) by
barmail.cs.umass.edu for <hg@cs.umass.edu>; Fri,
07 Nov 2008 13:27:07 -0500 (EST)
Received: from [58.88.21.177] by
inbnd55.exchangeddd.com; Sat, 8 Nov 2008 01:27:07 +0700
From: "Jonny" <tennis5@pp33head.com>
To: <hg@cs.umass.edu>
Subject: How to secure your savings

```

- P16. 阅读 POP3 的 RFC，即 RFC 1939。UIDL POP3 命令的目的是什么？
- P17. 考虑用 POP3 访问你的电子邮件。
- 假定你已经配置以下载并删除模式运行的 POP 邮件客户。完成下列事务：


```

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: blah blah ...
S: .....blah
S: .
?
?

```
 - 假定你已经配置以下载并保持模式运行的 POP 邮件客户。完成下列事务：


```

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: blah blah ...
S: .....blah
S: .
?
?

```
 - 假定你已经配置以下载并保持模式运行的 POP 邮件客户。使用（b）中的记录，假定你检索报文 1 和 2，退出 POP，5 分钟以后，你再访问 POP 以检索新电子邮件。假定在这 5 分钟间隔内，没有新报文发送给你。给出第二种 POP 会话的记录。

P18. 如题：

- 什么是 whois 数据库？
- 使用因特网上的各种 whois 数据库，获得两台 DNS 服务器的名字。指出你使用的是哪个 whois 数据库。
- 你本地机器上使用 nslookup 向 3 台 DNS 服务器发送 DNS 查询：你的本地 DNS 服务器和两台你在 (b) 中发现的 DNS 服务器。尝试对类型 A、NS 和 MX 报告进行查询。总结你的发现。
- 使用 nslookup 找出一台具有多个 IP 地址的 Web 服务器。你所在的机构（学校或公司）的 Web 服务器具有多个 IP 地址吗？
- 使用 ARIN whois 数据库，确定你所在大学使用的 IP 地址范围。
- 描述一个攻击者在发动攻击前，能够怎样利用 whois 数据库和 nslookup 工具来执行对一个机构的侦察。
- 讨论为什么 whois 数据库应当为公众所用。

P19. 在本习题中，我们使用在 Unix 和 Linux 主机上可用的 dig 工具来探索 DNS 服务器的等级结构。图 2-21 讲过，在 DNS 等级结构中较高的 DNS 服务器授权对该等级结构中较低 DNS 服务器的 DNS 请求，这是通过向 DNS 客户发送回那台较低层次的 DNS 服务器的名字来实现的。先阅读 dig 的帮助页，再回答下列问题。

- 从一台根 DNS 服务器（从根服务器 [a-m].root-server.net 之一）开始，通过使用 dig 得到你所在系的 Web 服务器的 IP 地址，发起一系列查询。显示回答你的查询的授权链中的 DNS 服务器的名字列表。
- 对几个流行 Web 站点如 google.com、yahoo.com 或 amazon.com，重复上一小题。

P20. 假定你能够访问所在系的本地 DNS 服务器中的缓存。你能够提出一种方法来粗略地确定在你所在系的用户中最为流行的 Web 服务器（你所在系以外）吗？解释原因。

P21. 假设你所在系具有一台用于系里所有计算机的本地 DNS 服务器。你是普通用户（即你不是网络/系统管理员）。你能够确定是否在几秒钟前从你系里的一台计算机可能访问过一台外部 Web 站点吗？解释原因。

P22. 考虑向 N 个对等方分发 $F = 15\text{Gb}$ 的一个文件。该服务器具有 $u_s = 30\text{Mbps}$ 的上载速率，每个对等方具有 $d_i = 2\text{Mbps}$ 的下载速率和上载速率 u 。对于 $N = 10$ 、100 和 1000 并且 $u = 300\text{kbps}$ 、700kbps 和 2Mbps，对于 N 和 u 的每种组合绘制出确定最小分发时间的图表。需要分别针对客户 - 服务器分发和 P2P 分发两种情况制作。

P23. 考虑使用一种客户 - 服务器体系结构向 N 个对等方分发一个 F 比特的文件。假定一种流体模型，即某服务器能够同时向多个对等方传输，只要组合速率不超过 u_s ，则以不同的速率向每个对等方传输。

- 假定 $u_s/N \leq d_{\min}$ 。定义一个具有 NF/u_s 分发时间的分发方案。
- 假定 $u_s/N \geq d_{\min}$ 。定义一个具有 $F/d_{\min}$ 分发时间的分发方案。
- 得出最小分发时间通常是由 $\max\{NF/u_s, F/d_{\min}\}$ 所决定的结论。

P24. 考虑使用 P2P 体系结构向 N 个用户分发 F 比特的一个文件。假定一种流体模型。为了简化起见，假定 $d_{\min}$ 很大，因此对等方下载带宽不会成为瓶颈。

- 假定 $u_s \leq (u_s + u_1 + \dots + u_N)/N$ 。定义一个具有 F/u_s 分发时间的分发方案。
- 假定 $u_s \geq (u_s + u_1 + \dots + u_N)/N$ 。定义一个具有 $NF/(u_s + u_1 + \dots + u_N)$ 分发时间的分发方案。
- 得出最小分发时间通常是由 $\max\{F/u_s, NF/(u_s + u_1 + \dots + u_N)\}$ 所决定的结论。

P25. 考虑在一个有 N 个活跃对等方的覆盖网络中，每对对等方有一条活跃的 TCP 连接。此外，假定该 TCP 连接通过总共 M 台路由器。在对应的覆盖网络中，有多少结点和边？

P26. 假定 Bob 加入 BitTorrent，但他不希望向任何其他对等方上载任何数据（因此称为搭便车）。

- Bob 声称他能够收到由该社区共享的某文件的完整副本。Bob 所言是可能的吗？为什么？
- Bob 进一步声称他还能够更为有效地进行他的“搭便车”，方法是利用所在系的计算机实验室中的多台计算机（具有不同的 IP 地址）。他怎样才能做到这些呢？

- P27. 在 2.6.2 节的环形 DHT 例子中，假定对等方 3 知道对等方 5 已经离开。对等方 3 如何更新它的后继状态信息？此时哪个对等方是它的第一个后继？哪个是其第二个后继？
- P28. 在 2.6.2 节的环形 DHT 例子中，假定一个新的对等方 6 要接入该 DHT，并且对等方 6 最初只知道对等方 15 的 IP 地址。需要采用哪些步骤？
- P29. 因为一个位于 $[0, 2^n - 1]$ 的整数能被标识为一个在 DHT 中的 n 比特的二进制数，每个键能被表示为 $k = (k_0, k_1, \dots, k_{n-1})$ ，并且每个对等方标识符能被表示为 $p = (p_0, p_1, \dots, p_{n-1})$ 。我们现在定义键 k 和对等方 p 的异或 (XOR) 距离为

$$d(k, p) = \sum_{j=0}^{n-1} |k_j - p_j| 2^j$$

描述该度量如何用于为对等方分配（键，值）对。（要学习如何使用这个天然的度量构建有效的 DHT，参见描述 Kademlia DHT 的文献 [Maymounkov 2002]。）

- P30. 由于 DHT 是覆盖网络，它们也许不必与底层的物理网络匹配得很好，即两个相邻的对等方也许物理上相距很远；例如，一个对等方可能位于亚洲而它的邻居可能位于北美。如果我们随机并统一地为新加入的对等方分配标识符，这个分配方案将会引起这种误匹配吗？揭示原因。这种误匹配如何影响 DHT 的性能呢？
- P31. 在一台主机上安装编译 TCPClient 和 UDPClient Python 程序，在另一台主机上安装编译 TCPServer 和 UDPServer 程序。
- 如果你在运行 TCPServer 之前运行 TCPClient，将发生什么现象？为什么？
 - 如果你在运行 UDPServer 之前运行 UDPClient，将发生什么现象？为什么？
 - 如果你对客户端和服务端使用了不同的端口，将发生什么现象？
- P32. 假定在 UDPClient.py 中在创建套接字后增加了下面一行：
- ```
clientSocket.bind(('', 5432))
```
- 有必要修改 UDPServer.py 吗？UDPClient 和 UDPServer 中的套接字端口号是多少？在变化之前它们是多少？
- P33. 你能够配置浏览器以打开对某 Web 站点的多个并行连接吗？有大量的并行 TCP 连接的优点和缺点是什么？
- P34. 我们已经看到因特网 TCP 套接字将数据处理为字节流，而 UDP 套接字识别报文边界。面向字节 API 与显式识别和维护应用程序定义的报文边界的 API 相比，试给出一个优点和一个缺点。
- P35. 什么是 Apache Web 服务器？它值多少钱？它当前有多少功能？为回答这个问题，你也许要看一下维基百科。
- P36. 许多 BitTorrent 客户使用 DHT 来创建一个分布式跟踪器。对于这些 DHT，“键”是什么，“值”是什么？



## 套接字编程作业

配套 Web 网站包括了 6 个套接字编程作业。前四个作业简述如下。第 5 个作业利用了 ICMP 协议，在第 4 章结尾简述。第 6 个作业使用了多媒体协议，在第 7 章结尾简述。极力推荐学生们完成这些作业中的几个（如果不是全部的话）。学生们能够在 Web 网站 <http://www.awl.com/kurose-ross> 上找到这些作业的全面细节，以及 Python 代码的重要片段。

### 作业 1：Web 服务器

在这个编程作业中，你将用 Python 语言开发一个简单的 Web 服务器，它仅能处理一个请求。具体而言，你的 Web 服务器将：（1）当一个客户（浏览器）联系时创建一个连接套接字；（2）从这个连接接收 HTTP 请求；（3）解释该请求以确定所请求的特定文件；（4）从服务器的文件系统获得请求的文件；（5）创建一个由请求的文件组成的 HTTP 响应报文，报文前面有首部行；（6）经 TCP 连接向请求的浏览



器发送响应。如果浏览器请求一个在该服务器中不存在的文件，服务器应当返回一个“404 Not Found”差错报文。

在配套网站中，我们提供了用于该服务器的框架代码。你的任务是完善该代码，运行服务器，通过在不同主机上运行的浏览器发送请求来测试该服务器。如果运行你服务器的主机上已经有一个 Web 服务器在运行，你应当为该 Web 服务器使用一个不同于 80 端口的其他端口。

### 作业 2：UDP ping 程序

在这个编程作业中，你将用 Python 编写一个客户 ping 程序。该客户将发送一个简单的 ping 报文，接收一个从服务器返回的对应 pong 报文，并确定从该客户发送 ping 报文到接收到 pong 报文为止的时延。该时延称为往返时延（RTT）。由该客户和服务器提供的功能类似于在现代操作系统中可用的标准 ping 程序。然而，标准的 ping 使用互联网控制报文协议（ICMP）（我们将在第 4 章中学习 ICMP）。此时我们将创建一个非标准（但简单）的基于 UDP 的 ping 程序。

你的 ping 程序经 UDP 向目标服务器发送 10 个 ping 报文。对于每个报文，当对应的 pong 报文返回时，你的客户要确定和打印 RTT。因为 UDP 是一个不可靠的协议，由客户发送的分组可能会丢失。为此，客户不能无限期地等待对 ping 报文的回答。客户等待服务器回答的时间至多为 1 秒；如果没有收到回答，客户假定该分组丢失并相应地打印一条报文。

在此作业中，将给出服务器的完整代码（在配套网站中可找到）。你的任务是编写客户代码，该代码与服务器代码非常类似。建议你先仔细学习服务器的代码，然后编写你的客户代码，可以不受限制地从服务器代码中剪贴代码行。

### 作业 3：邮件客户

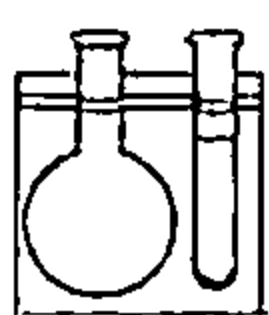
这个编程作业的目的是创建一个向任何接收方发送电子邮件的简单邮件客户。你的客户将必须与邮件服务器（如谷歌的电子邮件服务器）创建一个 TCP 连接，使用 SMTP 协议与邮件服务器进行交谈，经该邮件服务器向某接收方（如你的朋友）发送一个电子邮件报文，最后关闭与该邮件服务器的 TCP 连接。

对本作业，配套 Web 站点为你的客户提供了框架代码。你的任务是完善该代码并通过向不同的用户账户发送电子邮件来测试你的客户。你也可以尝试通过不同的服务器（例如谷歌的邮件服务器和你所在大学的邮件服务器）进行发送。

### 作业 4：多线程 Web 代理服务器

在这个编程作业中，你将研发一个简单的 Web 代理服务器。当你的代理服务器从一个浏览器接收到对某对象的 HTTP 请求，它生成对相同对象的一个新 HTTP 请求并向初始服务器发送。当该代理从初始服务器接收到具有该对象的 HTTP 响应时，它生成一个包括该对象的新 HTTP 响应，并发送给该客户。这个代理将是多线程的，使其在相同时间能够处理多个请求。

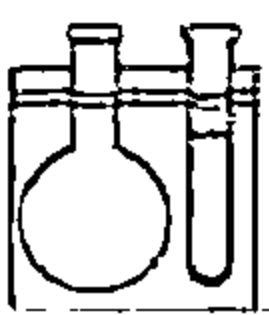
对本作业而言，配套 Web 网站对该代理服务器提供了框架代码。你的任务是完善该代码，然后测试你的代理，方法是让不同的浏览器经过你的代理来请求 Web 对象。



## Wireshark 实验：HTTP

在实验 1 中，我们已经初步使用了 Wireshark 分组嗅探器，我们现在准备使用 Wireshark 来研究运行中的协议。在本实验中，我们将研究 HTTP 协议的几个方面：基本的 GET/回答交互，HTTP 报文格式，检索大 HTML 文件，检索具有内嵌 URL 的 HTML 文件，持续和非持续连接，HTTP 鉴别和安全性。

如同所有的 Wireshark 实验一样，对该实验的全面描述可查阅本书的 Web 站点 <http://www.awl.com/kurose-ross>。



## Wireshark 实验：DNS

在本实验中，我们仔细观察 DNS 的客户端（DNS 是用于将因特网主机名转换为 IP 地址的协议）。



2.5 节讲过，在 DNS 的客户角色是相当简单的：客户向它的本地 DNS 服务器发送一个请求，并接收返回的响应。在此过程中发生的很多事情均不为 DNS 客户所见，如等级结构的 DNS 服务器互相通信递归地或迭代地解析该客户的 DNS 请求。然而，从 DNS 客户的角度而言，该协议是相当简单的，即向本地 DNS 服务器发送一个请求，从该服务器接收一个响应。在本实验中我们观察运转中的 DNS。

如同所有的 Wireshark 实验一样，在本书的 Web 站点 <http://www.awl.com/kurose-ross> 可以找到本实验的完整描述。

## 人物专访

Marc Andreessen 是 Mosaic 的共同发明人，Mosaic 是一种 Web 浏览器，正是它使万维网在 1993 年流行起来。Mosaic 具有一个清晰、易于理解的界面，是首个能嵌在文本中显示图像的浏览器。在 1994 年，Marc Andreessen 和 Jim Clark 创办了 Netscape 公司，其浏览器是到 20 世纪 90 年代中期为止最为流行的。Netscape 也研发了安全套接字层（SSL）协议和许多因特网服务器产品，包括邮件服务器和基于 SSL 的 Web 服务器。他现在是风险投资公司 Andreessen Horowitz 的共同奠基人和一般股东，对包括 Facebook、Foursquare、Groupon、Jawbone、Twitter 和 Zynga 等公司的财产投资搭配进行监管。他服务于包括 Bump、eBay、Glam Media、Facebook 和 HP 等在内的多个董事会。他具有美国伊利诺伊大学厄巴纳 - 香槟分校的计算机科学理学学士学位。



Marc Andreessen

• 您是怎样变得对计算感兴趣的？您过去一直知道您要从事信息技术吗？

在我长大成人的过程中，视频游戏和个人计算正好成为成功而风行一时的事物，在 20 世纪 70 年代后期和 80 年代初期，个人计算成为了新技术发展前沿。那时不只有苹果和 IBM 的个人计算机，而且有如 Commodore 和 Atari 等数以百计的新公司。我在 10 岁时用一本名为《简明 BASIC 速成》（Instant Freeze-Dried BASIC）的书进行自学，并在 12 岁时得到自己的第一台计算机（TRS-80 Color Computer，查查它！）。

• 请描述您职业生涯中干过的最令人激动的一两个项目。最大的挑战是什么？

毋庸置疑，最令人兴奋的项目是 1992 ~ 1993 年的初始 Mosaic Web 浏览器，最大的挑战是让任何人从此往后都认真地对待它。在那个时候，每个人都认为交互式未来将是由大型公司宣布的“交互式电视”，而非由新兴公司发明的因特网。

• 您对网络和因特网未来的什么东西感到兴奋？您最为关注什么？

最为兴奋的东西是程序员和企业家能够开发的应用和服务的巨大的尚待开发的领域，即因特网已经释放的创造性到达了一种我们以前从未预见到的水平。我最关注是“意想不到的后果”的原则，即我们并不总是知道我们所做事情后果，例如因特网被政府所用，使得监视居民到达了一种新水平。

• 随着 Web 技术的进展，学生们有什么应当特别要了解的？

改变的速度，即对学习来说，最重要的东西是学习的方法，在特定的技术中如何灵活地适应改变，当你在职业生涯中前行时，在新的机会和可能性方面如何保持开放的思想。

• 什么人给予您专业灵感？

他们是：Vannevar Bush, Ted Nelson, Doug Engelbart, Nolan Bushnell, Bill Hewlett 和 Dave Packard, Ken Olsen, Steve Jobs, Steve Wozniak, Andy Grove, Grace Hopper, Hedy Lamarr, Alan Turing, Richard Stallman。

• 对于要在计算和信息技术领域谋求发展的学生们，您有什么忠告？

尽可能深入地理解技术是怎样创造的，然后补充学习商业运作的原理。

• 技术能够解决世界的问题吗？

不能，但是通过经济增长我们推动人们生活标准的改善。综观历史，大多数经济增长来自技术，因此就像技术带来的好处一样。



# 运 输 层

运输层位于应用层和网络层之间，是分层的网络体系结构的重要部分。该层为运行在不同主机上的应用进程提供直接的通信服务起着至关重要的作用。我们在本章采用的教学方法是，交替地讨论运输层的原理和这些原理在现有的协议中是如何实现的。与往常一样，我们将特别关注因特网协议，即 TCP 和 UDP 运输层协议。

我们将从讨论运输层和网络层的关系开始。这就进入了研究运输层第一个关键功能的阶段，即将网络层的在两个端系统之间的交付服务扩展到运行在两个不同端系统上的应用层进程之间的交付服务。我们将在讨论因特网的无连接运输协议 UDP 时阐述这个功能。

然后我们重新回到原理学习上，面对计算机网络中最为基础性的问题之一，即两个实体怎样才能以一种会丢失或损坏数据的媒体上可靠地通信。通过一系列复杂性不断增加（从而更真实！）的情况，我们将逐步建立起一套被运输协议用来解决这些问题的技术。然后，我们将说明这些原理是如何体现在因特网面向连接的运输协议 TCP 中的。

接下来我们讨论网络中的第二个基础性的重大问题，即控制运输层实体的传输速率以避免网络中的拥塞，或从拥塞中恢复过来。我们将考虑拥塞的原因和后果，以及常用的拥塞控制技术。在透彻地理解了拥塞控制问题之后，我们将研究 TCP 的拥塞控制方法。

## 3.1 概述和运输层服务

在前两章中，我们已对运输层的作用及其所提供的服务有所了解。现在我们快速地回顾一下前面学过的有关运输层的知识。

运输层协议为运行在不同主机上的应用进程之间提供了**逻辑通信**（logic communication）功能。从应用程序的角度看，通过逻辑通信，运行不同进程的主机好像直接相连一样；实际上，这些主机也许位于地球的两侧，通过很多路由器及多种不同类型的链路相连。应用进程使用运输层提供的逻辑通信功能彼此发送报文，而无需考虑承载这些报文的物理基础设施的细节。图 3-1 图示了逻辑通信的概念。

如图 3-1 所示，运输层协议是在端系统中而不是在路由器中实现的。在发送端，运输层将从发送应用程序进程接收到的报文转换成运输层分组，用因特网术语来讲该分组称为**运输层报文段**（segment）。实现的方法（可能）是将应用报文划分为较小的块，并为每块加上一个运输层首部以生成运输层报文段。然后，在发送端系统中，运输层将这些报文段传递给网络层，网络层将其封装成网络层分组（即数据报）并向目的地发送。注意到下列事实是重要的：网络路由器仅作用于该数据报的网络层字段；即它们不检查封装在该数据报的运输层报文段的字段。在接收端，网络层从数据报中提取运输层报文段，并将该报文段向上交给运输层。运输层则处理接收到的报文段，使该报文段中的数据为接收应用进程使用。

网络应用程序可以使用多种的运输层协议。例如，因特网有两种协议，即 TCP 和 UDP。每种协议都能为调用的应用程序提供一组不同的运输层服务。



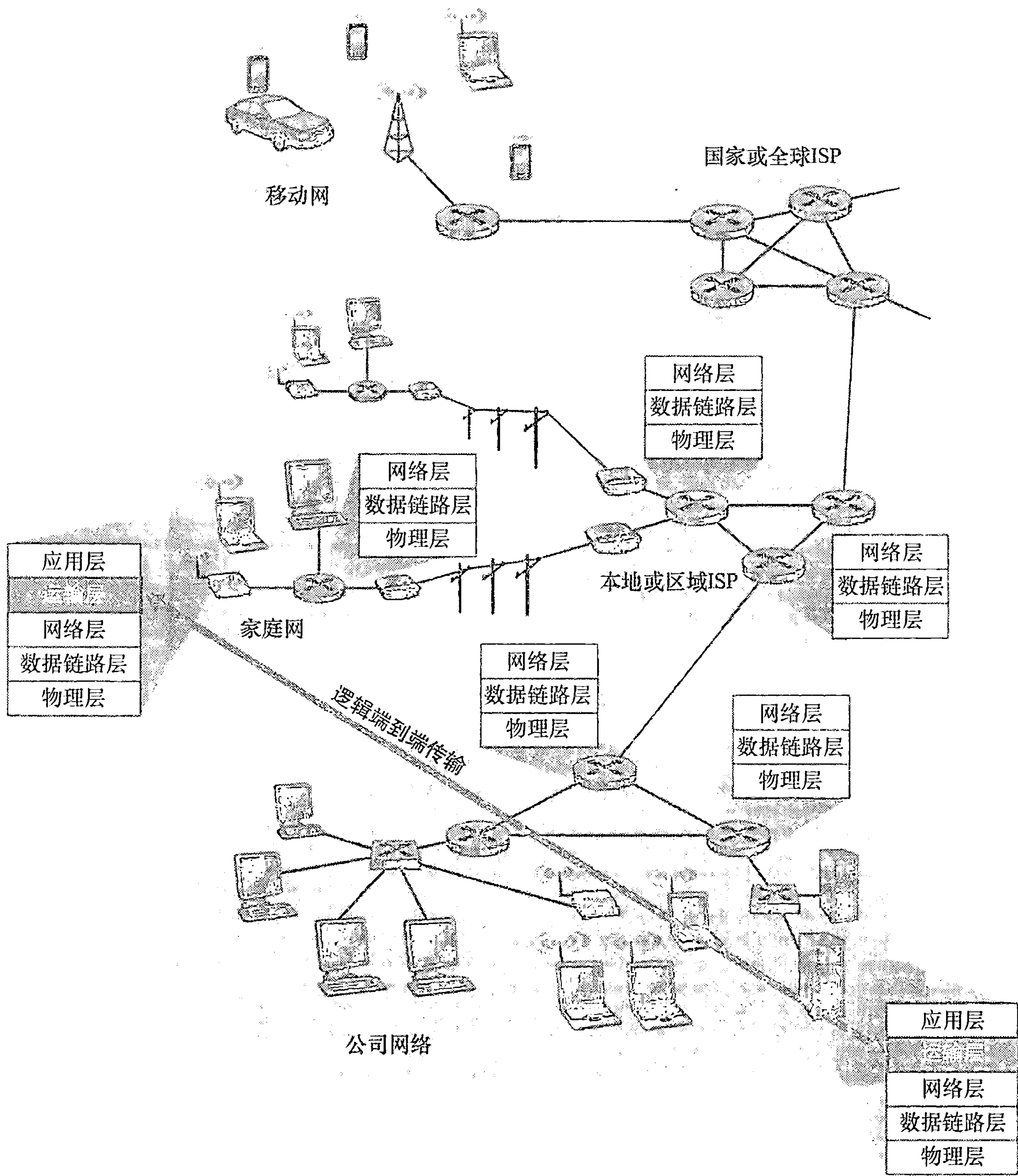


图 3-1 运输层在应用程序进程间提供逻辑的而非物理的通信

3. 1. 1 运输层和网络层的关系

前面讲过，在协议栈中，运输层刚好位于网络层之上。网络层提供了主机之间的逻辑通信，而运输层为运行在不同主机上的进程之间提供了逻辑通信。这种差别虽然细微但很重要。我们用一个家庭类比来帮助分析这种差别。

考虑有两个家庭，一家位于美国东海岸，一家位于美国西海岸，每家有 12 个孩子。东海岸家庭的孩子们是西海岸家庭孩子们的堂兄弟姐妹。这两个家庭的孩子们喜欢彼此通信，每个人每星期要互相写一封信，每封信都用单独的信封通过传统的邮政服务传送。因此，每个家庭每星期向另一家发送 144 封信。（如果有电子邮件的话，这些孩子可以省不少钱！）每一个家庭有个孩子负责收发邮件，西海岸家庭是 Ann 而东海岸家庭是 Bill。



每星期 Ann 去她的所有兄弟姐妹那里收集信件，并将这些信件交到每天到家门口来的邮政服务的邮车上。当信件到达西海岸家庭时，Ann 也负责将信件分发到她的兄弟姐妹手上。在东海岸家庭中的 Bill 也负责类似的工作。

在这个例子中，邮政服务为两个家庭间提供逻辑通信，邮政服务将信件从一家送往另一家，而不是从一个人送往另一个人。在另一方面，Ann 和 Bill 为堂兄弟姐妹之间提供了逻辑通信，Ann 和 Bill 从兄弟姐妹那里收取信件或到兄弟姐妹那里交付信件。注意到从堂兄弟姐妹们的角度来看，Ann 和 Bill 就是邮件服务，尽管他们只是端到端交付过程的一部分（即端系统部分）。在解释运输层和网络层之间的关系时，这个家庭的例子是一个非常好的类比。

应用层报文 = 信封上的字符

进程 = 堂兄弟姐妹

主机(又称为端系统) = 家庭

运输层协议 = Ann 和 Bill

网络层协议 = 邮政服务(包括邮车)

我们继续观察这个类比。值得注意的是，Ann 和 Bill 都是在各自家里进行工作的；例如，他们并没有参与任何一个中间邮件中心对邮件进行分拣，或者将邮件从一个邮件中心送到另一个邮件中心之类的工作。类似地，运输层协议只工作在端系统中。在端系统中，运输层协议将来自应用进程的报文移动到网络边缘（即网络层），反过来也是一样，但对有关这些报文在网络核心如何移动并不作任何规定。事实上，如图 3-1 所示，中间路由器既不处理也不识别运输层加在应用层报文的任何信息。

我们还是继续讨论这两家的情况。现在假定 Ann 和 Bill 外出度假，另外一对堂兄妹（如 Susan 和 Harvey）接替他们的工作，在家庭内部进行信件的收集和交付工作。不幸的是，Susan 和 Harvey 的收集和交付工作与 Ann 和 Bill 所做的并不完全一样。由于年龄更小，Susan 和 Harvey 收发邮件的次数更少，而且偶尔还会丢失邮件（有时是被家里的狗咬坏了）。因此，Susan 和 Harvey 这对堂兄妹并没有提供与 Ann 和 Bill 一样的服务集合（即相同的服务模型）。与此类似，计算机网络中可以安排多种运输层协议，每种协议为应用程序提供不同的服务模型。

Ann 和 Bill 所能提供的服务明显受制于邮政服务所能提供的服务。例如，如果邮政服务不能提供在两家之间传递邮件所需时间的最长期限（例如 3 天），那么 Ann 和 Bill 就不可能保证邮件在堂兄弟姐妹之间传递信件的最长期限。与此类似，运输协议能够提供的服务常常受制于底层网络层协议的服务模型。如果网络层协议无法为主机之间发送的运输层报文段提供时延或带宽保证的话，运输层协议也就无法为进程之间发送的应用程序报文提供时延或带宽保证。

然而，即使底层网络协议不能在网络层提供相应的服务，运输层协议也能提供某些服务。例如，如我们将在本章所见，即使底层网络协议是不可靠的，也就是说网络层协议会使分组丢失、篡改和冗余，运输协议也能为应用程序提供可靠的数据传输服务。另一个例子是（我们在第 8 章讨论网络安全时将会研究到），即使网络层不能保证运输层报文段的机密性，运输协议也能使用加密来确保应用程序报文不被入侵者读取。

### 3.1.2 因特网运输层概述

前面讲过因特网（更一般地讲是一个 TCP/IP 网络）为应用层提供了两种截然不同的



可用运输层协议。这些协议一种是 UDP（用户数据报协议），它为调用它的应用程序提供了一种不可靠、无连接的服务。另一种是 TCP（传输控制协议），它为调用它的应用程序提供了一种可靠的、面向连接的服务。当设计一个网络应用程序时，该应用程序的开发人员必须指定使用这两种运输协议中的哪一种。如我们在 2.7 节看到的那样，应用程序开发人员在生成套接字时必须指定是选择 UDP 还是选择 TCP。

为了简化术语，在与因特网有关的环境中，我们将运输层分组称为报文段（segment）。然而，需要指出的是，因特网文献（如 RFC 文档）也将 TCP 的运输层分组称为报文段，而常将 UDP 的分组称为数据报。而这类因特网文献也将网络层分组称为数据报！本书作为一本计算机网络的入门书籍，我们认为将 TCP 和 UDP 的分组统称为报文段，而将数据报名称保留给网络层分组不容易混淆。

在对 UDP 和 TCP 进行简要介绍之前，简单介绍一下因特网的网络层（我们将在第 4 章中详细地学习网络层）是有用的。因特网网络层协议有一个名字叫 IP，即网际协议。IP 为主机之间提供了逻辑通信。IP 的服务模型是尽力而为交付服务（best-effort delivery service）。这意味着 IP 尽它“最大的努力”在通信的主机之间交付报文段，但它并不做任何确保。特别是，它不确保报文段的交付，不保证报文段的按序交付，不保证报文段中数据的完整性。由于这些原因，IP 被称为不可靠服务（unreliable service）。在此还要指出的是，每台主机至少有一个网络层地址，即所谓的 IP 地址。我们在第 4 章将详细讨论 IP 地址；在这一章中，我们只需要记住每台主机有一个 IP 地址。

在对 IP 服务模型有了初步了解后，我们总结一下 UDP 和 TCP 所提供的服务模型。UDP 和 TCP 最基本的责任是，将两个端系统间 IP 的交付服务扩展为运行在端系统上的两个进程之间的交付服务。将主机间交付扩展到进程间交付被称为运输层的多路复用（transport-layer multiplexing）与多路分解（demultiplexing）。我们将在下一节讨论运输层的多路复用与多路分解。UDP 和 TCP 还可以通过在其报文段首部中包括差错检查字段而提供完整性检查。进程到进程的数据交付和差错检查是两种最低限度的运输层服务，也是 UDP 所能提供的仅有的两种服务。特别是，与 IP 一样，UDP 也是一种不可靠的服务，即不能保证一个进程所发送的数据能够完整无缺地（或全部！）到达目的进程。在 3.3 节中将更详细地讨论 UDP。

另一方面，TCP 为应用程序提供了几种附加服务。首先，它提供可靠数据传输（reliable data transfer）。通过使用流量控制、序号、确认和定时器（本章将详细介绍这些技术），TCP 确保正确地、按序地将数据从发送进程交付给接收进程。这样，TCP 就将两个端系统间的不可靠 IP 服务转换成了一种进程间的可靠数据传输服务。TCP 还提供拥塞控制（congestion control）。拥塞控制与其说是一种提供给调用它的应用程序的服务，不如说是一种提供给整个因特网的服务，这是一种带来通用好处的服务。不太严格地说，TCP 拥塞控制防止任何一条 TCP 连接用过多流量来淹没通信主机之间的链路和交换设备。TCP 力求为每个通过一条拥塞网络链路的连接平等地共享网络链路带宽。这可以通过调节 TCP 连接的发送端发送进网络的流量速率来做到。在另一方面，UDP 流量是不可调节的。使用 UDP 传输的应用程序可以根据其需要以其愿意的任何速率发送数据。

一个能提供可靠数据传输和拥塞控制的协议必定是复杂的。我们将用几节的篇幅来介绍可靠数据传输和拥塞控制的原理，用另外几节介绍 TCP 协议本身。3.4 ~ 3.8 节将研究这些主题。本章采取基本原理和 TCP 协议交替介绍的方法。例如，我们首先在一般环境下



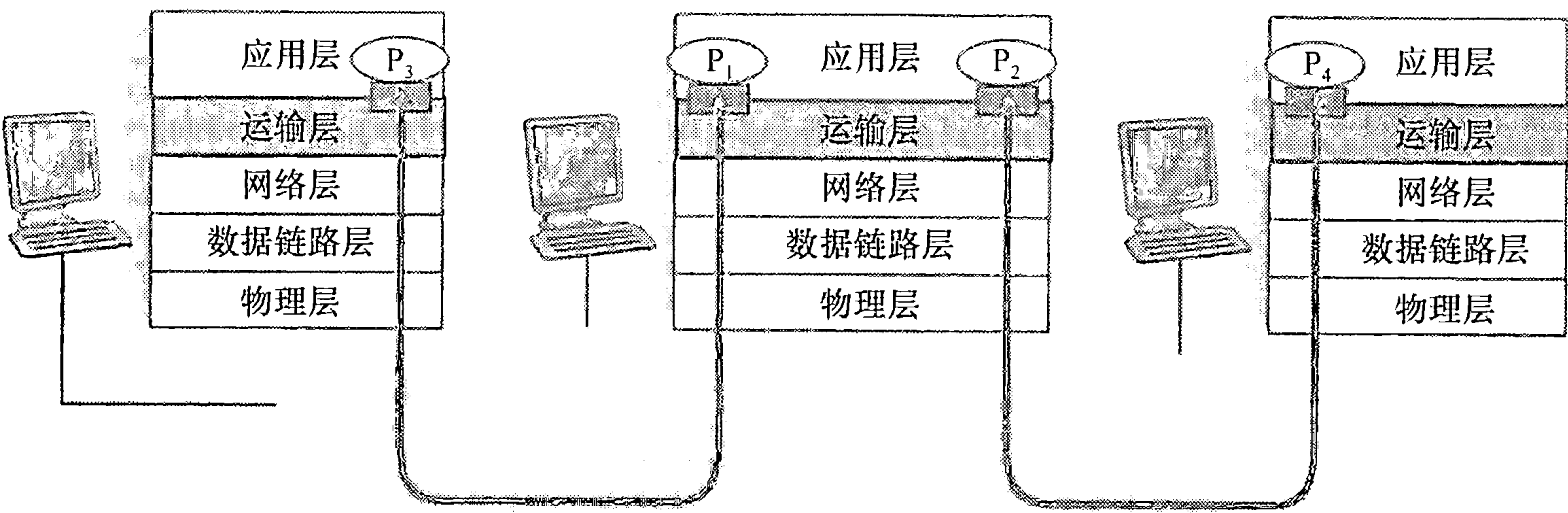
讨论可靠数据传输，然后讨论 TCP 是怎样具体提供可靠数据传输的。类似地，先在一般环境下讨论拥塞控制，然后讨论 TCP 是怎样实现拥塞控制的。但在全面介绍这些内容之前，我们先学习运输层的多路复用与多路分解。

3.2 多路复用与多路分解

在本节中，我们讨论运输层的多路复用与多路分解，也就是将由网络层提供的主机到主机交付服务延伸到为运行在主机上的应用程序提供进程到进程的交付服务。为了使讨论具体起见，我们将在因特网环境中讨论这种基本的运输层服务。然而，需要强调的是，多路复用与多路分解服务是所有计算机网络都需要的。

在目的主机，运输层从紧邻其下的网络层接收报文段。运输层负责将这些报文段中的数据交付给在主机上运行的适当应用程序进程。我们来看一个例子。假定你正坐在计算机前下载 Web 页面，同时还在运行一个 FTP 会话和两个 Telnet 会话。这样你就有 4 个网络应用进程在运行，即两个 Telnet 进程，一个 FTP 进程和一个 HTTP 进程。当你的计算机中的运输层从底层的网络层接收数据时，它需要将所接收到的数据定向到这 4 个进程中的一个。现在我们来研究这是怎样完成的。

首先回想 2.7 节的内容，一个进程（作为网络应用的一部分）有一个或多个套接字 (socket)，它相当于从网络向进程传递数据和从进程向网络传递数据的门户。因此，如图 3-2 所示，在接收主机中的运输层实际上并没有直接将数据交付给进程，而是将数据交给了一个中间的套接字。由于在任一时刻，在接收主机上可能有不止一个套接字，所以每个套接字都有唯一的标识符。标识符的格式取决于它是 UDP 还是 TCP 套接字，我们将很快对它们进行讨论。



图例：  
○ 进程    ■ 套接字

图 3-2 运输层的多路复用与多路分解

现在我们考虑接收主机怎样将一个到达的运输层报文段定向到适当的套接字。为此目的，每个运输层报文段中具有几个字段。在接收端，运输层检查这些字段，标识出接收套接字，进而将报文段定向到该套接字。将运输层报文段中的数据交付到正确的套接字的工作称为多路分解 (demultiplexing)。在源主机从不同套接字中收集数据块，并为每个数据块封装上首部信息（这将在以后用于分解）从而生成报文段，然后将报文段传递到网络



层，所有这些工作称为多路复用（multiplexing）。值得注意的是，图 3-2 中的中间那台主机的运输层必须将从其下的网络层收到的报文段分解后交给其上的  $P_1$  或  $P_2$  进程；这一过程是通过将到达的报文段数据定向到对应进程的套接字来完成的。中间主机中的运输层也必须收集从这些套接字输出的数据，形成运输层报文段，然后将其向下传递给网络层。尽管我们在因特网运输层协议的环境下引入了多路复用和多路分解，认识到下列事实是重要的：它们与在某层（在运输层或别处）的单一协议何时被位于接下来的较高层的多个协议使用有关。

为了说明分解的工作过程，可以再以前面一节的家庭进行类比。每一个孩子通过他们的名字来标识。当 Bill 从邮递员处收到一批信件，并通过查看收信人名字而将信件亲手交付给他的兄弟姐妹们时，他执行的就是一个分解操作。当 Ann 从兄弟姐妹们那里收集信件并将它们交给邮递员时，她执行的就是一个多路复用操作。

既然我们理解了运输层多路复用与多路分解的作用，那就再来看看在主机中它们实际是怎样工作的。通过上述讨论，我们知道运输层多路复用要求：①套接字有唯一标识符；②每个报文段有特殊字段来指示该报文段所要交付到的套接字。如图 3-3 所示，这些特殊字段是源端口号字段（source port number field）和目的端口号字段（destination port number field）。（UDP 报文段和 TCP 报文段还有其他的一些字段，这些将在本章后继几节中进行讨论。）端口号是一个 16 比特的数，其大小在 0 ~ 65535 之间。0 ~ 1023 范围的端口号称为周知端口号（well-known port number），是受限制的，这是指它们保留给诸如 HTTP（它使用端口号 80）和 FTP（它使用端口号 21）之类的周知应用层协议来使用。周知端口的列表在 RFC 1700 中给出，同时在 <http://www.iana.org> 上有更新文档 [RFC 3232]。当我们开发一个新的应用程序时（如在 2.7 节中开发的一个应用程序），必须为其分配一个端口号。

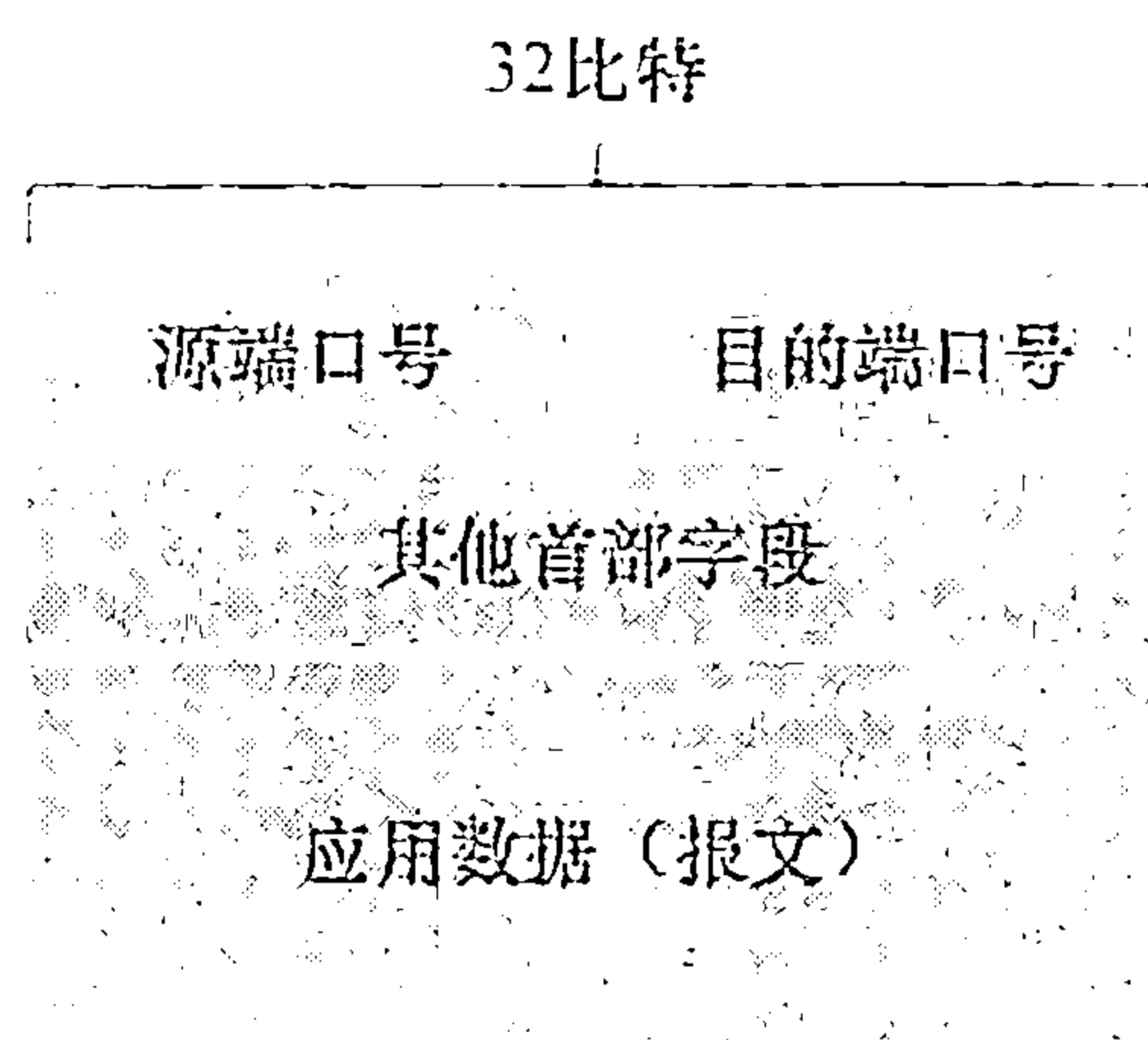


图 3-3 运输层报文段中的源与目的端口字段

现在应该清楚运输层是怎样能够实现分解服务的了：在主机上的每个套接字能够分配一个端口号，当报文段到达主机时，运输层检查报文段中的目的端口号，并将其定向到相应的套接字。然后报文段中的数据通过套接字进入其所连接的进程。如我们将看到的那样，UDP 基本上是这样做的。然而，也将如我们所见，TCP 中的多路复用与多路分解更为复杂。

### 1. 无连接的多路复用与多路分解

2.7.1 节讲过，在主机上运行的 Python 程序使用下面一行代码创建了一个 UDP 套接字：

```
clientSocket = socket(socket.AF_INET, socket.SOCK_DGRAM)
```

当用这种方式创建一个 UDP 套接字时，运输层自动地为该套接字分配一个端口号。特别是，运输层从范围 1024 ~ 65535 内分配一个端口号，该端口号是当前未被该主机中任何其他 UDP 端口使用的号。另外一种方法是，在创建一个套接字后，我们能够在 Python 程序中增加一行代码，通过套接字 bind() 方法为这个 UDP 套接字关联一个特定的端口号（如 19157）：

```
clientSocket.bind(('', 19157))
```



如果应用程序开发者所编写的代码实现的是一个“周知协议”的服务器端，那么开发者就必须为其分配一个相应的周知端口号。通常，应用程序的客户端让运输层自动地（并且是透明地）分配端口号，而服务器端则分配一个特定的端口号。

通过为 UDP 套接字分配端口号，我们现在能够精确地描述 UDP 的复用与分解了。假定在主机 A 中的一个进程具有 UDP 端口 19157，它要发送一个应用程序数据块给位于主机 B 中的另一进程，该进程具有 UDP 端口 46428。主机 A 中的运输层创建一个运输层报文段，其中包括应用程序数据、源端口号（19157）、目的端口号（46428）和两个其他值（将在后面讨论，它对当前的讨论并不重要）。然后，运输层将得到的报文段传递到网络层。网络层将该报文段封装到一个 IP 数据报中，并尽力而为地将报文段交付给接收主机。如果该报文段到达接收主机 B，接收主机运输层就检查该报文段中的目的端口号（46428）并将该报文段交付给端口号 46428 所标识的套接字。值得注意的是，主机 B 能够运行多个进程，每个进程有自己的 UDP 套接字及相应的端口号。值得注意的是，主机 B 可能运行多个进程，每个进程都具有其自己的 UDP 套接字和相联系的端口号。当从网络到达 UDP 报文段时，主机 B 通过检查该报文段中的目的端口号，将每个报文段定向（分解）到相应的套接字。

注意到下述事实是重要的：一个 UDP 套接字是由一个二元组来全面标识的，该二元组包含一个目的 IP 地址和一个目的端口号。因此，如果两个 UDP 报文段有不同的源 IP 地址和/或源端口号，但具有相同的目的 IP 地址和目的端口号，那么这两个报文段将通过相同的套接字被定向到相同的进程。

你也许现在想知道，源端口号的用途是什么呢？如图 3-4 所示，在 A 到 B 的报文段中，源端口号用作“返回地址”的一部分，即当 B 需要回发一个报文段给 A 时，B 到 A 的报文段中的目的端口号便从 A 到 B 的报文段中的源端口号中取值。（完整的返回地址是 A 的 IP 地址和源端口号。）举一个例子，回想 2.7 节学习过的那个 UDP 服务器程序。在 UDPServer.py 中，服务器使用 `recvfrom()` 方法从其自客户接收到的报文段中提取出客户端（源）端口号，然后，它将所提取的源端口号作为目的端口号，向客户发送一个新的报文段。

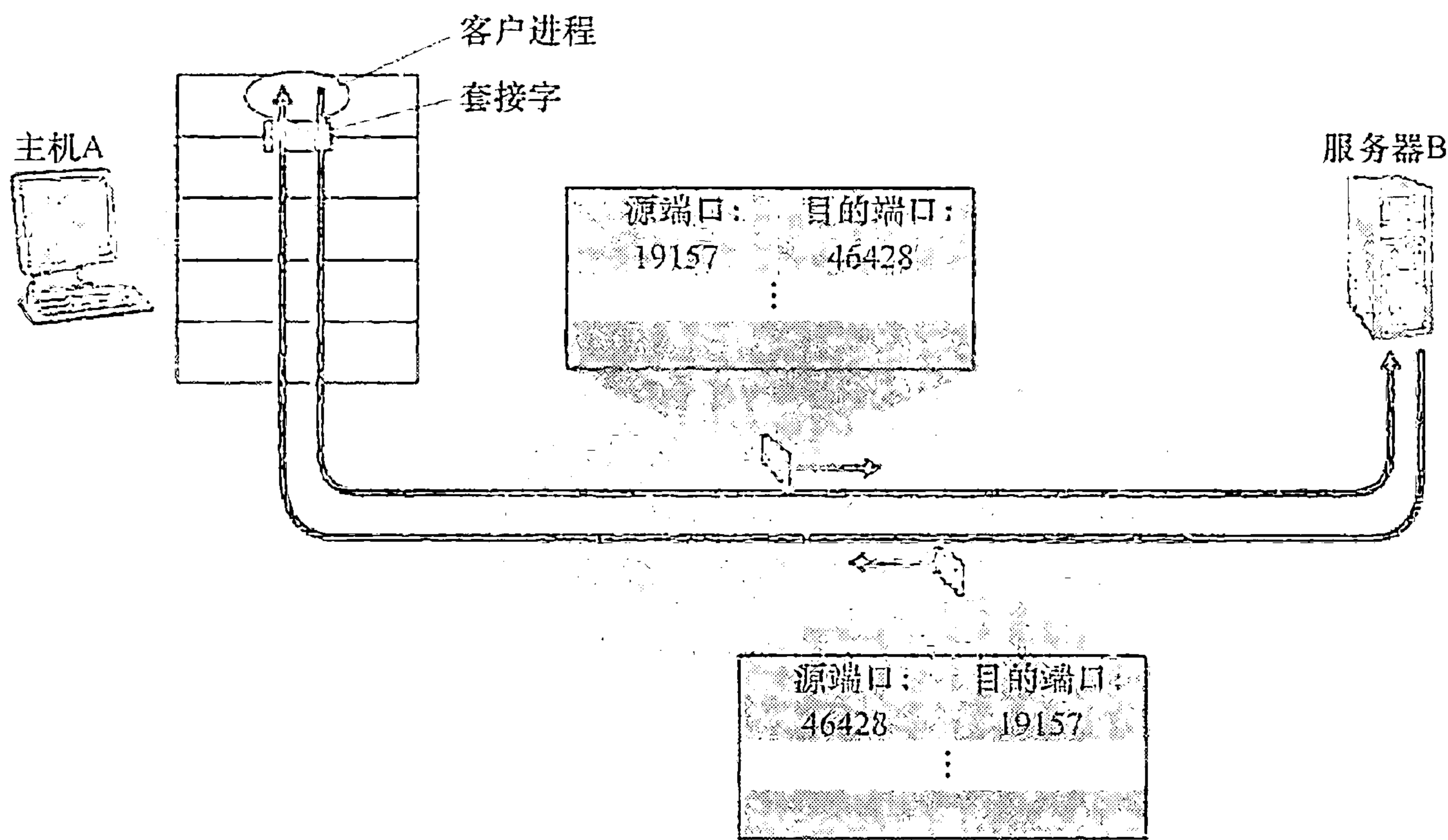


图 3-4 源端口号与目的端口号的反转



## 2. 面向连接的多路复用与多路分解

为了理解 TCP 分解，我们必须更为仔细地研究 TCP 套接字和 TCP 连接创建。TCP 套接字和 UDP 套接字之间的一个细微差别是，TCP 套接字是由一个四元组（源 IP 地址，源端口号，目的 IP 地址，目的端口号）来标识的。这样，当一个 TCP 报文段从网络到达一台主机时，该主机使用全部 4 个值来将报文段定向（分解）到相应的套接字。特别与 UDP 不同的是，两个具有不同源 IP 地址或源端口号的到达 TCP 报文段将被定向到两个不同的套接字，除非 TCP 报文段携带了初始创建连接请求。为了深入地理解这一点，我们再来重新考虑 2.7.2 节中的 TCP 客户 - 服务器编程的例子：

- TCP 服务器应用程序有一个“welcoming socket”，它在 12000 号端口上等待来自 TCP 客户（见图 2-29）的连接建立请求。

- TCP 客户使用下面的代码创建一个套接字并发送一个连接建立请求报文段：

```
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, 12000))
```

- 一条连接建立请求只不过是一个目的端口号为 12000，TCP 首部的特定“连接建立位”置位的 TCP 报文段（在 3.5 节进行讨论）。这个报文段也包含一个由客户选择的源端口号。

- 当运行服务器进程的计算机的主机操作系统接收到具有目的端口 12000 的连接请求报文段后，它就定位服务器进程，该进程正在端口号 12000 等待接受连接。该服务器进程则创建一个新的套接字：

```
connectionSocket, addr = serverSocket.accept()
```

- 该服务器的运输层还注意到连接请求报文段中的下列 4 个值：①该报文段中的源端口号；②源主机 IP 地址；③该报文段中的目的端口号；④自身的 IP 地址。新创建的连接套接字通过这 4 个值来标识。所有后续到达的报文段，如果它们的源端口号、源主机 IP 地址、目的端口号和目的 IP 地址都与这 4 个值匹配，则被分解到这个套接字。随着 TCP 连接完成，客户和服务端便可相互发送数据了。

服务器主机可以支持很多并行的 TCP 套接字，每个套接字与一个进程相联系，并由其四元组来标识每个套接字。当一个 TCP 报文段到达主机时，所有 4 个字段（源 IP 地址，源端口，目的 IP 地址，目的端口）被用来将报文段定向（分解）到相应的套接字。

### 关注安全性

#### 端口扫描

我们已经看到一个服务器进程潜在地在一个打开的端口等待远程客户的接触。某些端口为周知应用程序（例如 Web、FTP、DNS 和 SMTP 服务器）所预留；依照惯例其他端口由流行应用程序（例如微软 2000 SQL 服务器在 UDP 1434 端口上监听请求）使用。因此，如果我们确定一台主机上打开了一个端口，也许就能够将该端口映射到在该主机运行的一个特定的应用程序上。这对于系统管理员非常有用，系统管理员通常希望知晓有什么样的网络应用程序正运行在他们的网络主机上。而攻击者为了“寻找突破口”，也要知道在目标主机上有哪些端口打开。如果发现一台主机正在运行具有已知安全缺陷的应用程序（例如，在端口 1434 上监听的一台 SQL 服务器会遭受缓存溢出，使得一个



远程用户能在易受攻击的主机上执行任意代码，这是一种由 Slammer 蠕虫所利用的缺陷 [CERT 2003-04])，那么该主机已成为攻击者的囊中之物了。

确定哪个应用程序正在监听哪些端口是一件相对容易的事情。事实上有许多公共域程序（称为端口扫描器）做的正是这种事情。也许它们之中使用最广泛的是 nmap，该程序在 <http://nmap.org/> 上免费可用，并且包括在大多数 Linux 分发软件中。对于 TCP，nmap 顺序地扫描端口，寻找能够接受 TCP 连接的端口。对于 UDP，nmap 也是顺序地扫描端口，寻找对传输的 UDP 报文段进行响应的 UDP 端口。在这两种情况下，nmap 返回打开的、关闭的或不可达的端口列表。运行 nmap 的主机能够尝试扫描因特网中任何地方的目的主机。我们将在 3.5.6 节中再次用到 nmap，在该节中我们将讨论 TCP 连接管理。

图 3-5 图示了这种情况，图中主机 C 向服务器 B 发起了两个 HTTP 会话，主机 A 向服务器 B 发起了一个 HTTP 会话。主机 A 与主机 C 及服务器 B 都有自己唯一的 IP 地址，它们分别是 A、C、B。主机 C 为其两个 HTTP 连接分配了两个不同的源端口号（26145 和 7532）。因为主机 A 选择源端口号时与主机 C 互不相干，因此它也可以将源端口号 26145 分配给其 HTTP 连接。但这不是问题，即服务器 B 仍然能够正确地分解这两个具有相同源端口号的连接，因为这两条连接有不同的源 IP 地址。

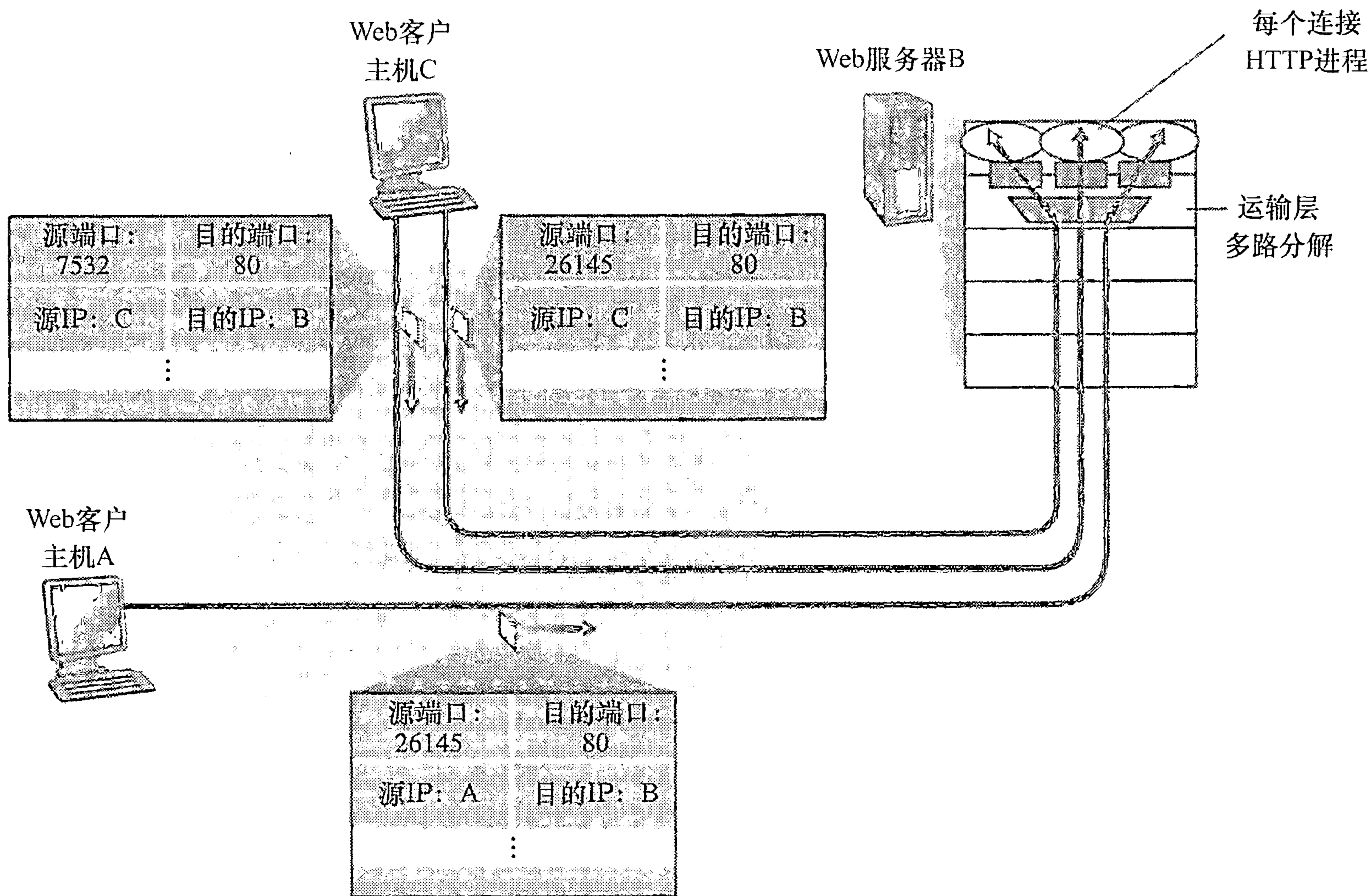


图 3-5 两个客户使用相同的端口号（80）与同一个 Web 服务器应用通信

### 3. Web 服务器与 TCP

在结束这个讨论之前，再多说几句 Web 服务器以及它们如何使用端口号是有益的。考虑一台运行 Web 服务器的主机，例如在端口 80 上运行一个 Apache Web 服务器。当客户



(如浏览器)向该服务器发送报文段时,所有报文段的端口都为80。特别是,初始连接建立报文段和承载HTTP请求的报文段都有80的端口。如我们刚才描述的那样,该服务器能够根据源IP地址和源端口号来区分来自不同客户的报文段。

图3-5显示了一台Web服务器为每条连接生成一个新进程。如图3-5所示,每个这样的进程都有自己的连接套接字,通过这些套接字可以收到HTTP请求和发送HTTP响应。然而,我们要提及的是,连接套接字与进程之间并非总是有着一一对应的关系。事实上,当今的高性能Web服务器通常只使用一个进程,但是为每个新的客户连接创建一个具有新连接套接字的新线程。(线程可被看作是一个轻量级的子进程。)如果做了第2章的第一个编程作业,你所构建的Web服务器就是这样工作的。对于这样一台服务器,在任意给定的时间内都可能有(具有不同标识的)许多连接套接字连接到相同的进程。

如果客户与服务器使用持续HTTP,则在整条连接持续期间,客户与服务器之间经由同一个服务器套接字交换HTTP报文。然而,如果客户与服务器使用非持续HTTP,则对每一对请求/响应都创建一个新的TCP连接并在随后关闭,因此对每一对请求/响应创建一个新的套接字并在随后关闭。这种套接字的频繁创建和关闭会严重地影响一个繁忙的Web服务器的性能(尽管有许多操作系统技巧可用来减轻这个问题的影响)。读者若对与持续和非持续HTTP有关的操作系统问题感兴趣的话,可参见[Nielsen 1997, Nahum 2002]。

既然我们已经讨论过了运输层多路复用与多路分解问题,下面我们就继续讨论因特网运输层协议之一,即UDP。在下一节中,我们将看到UDP无非就是对网络层协议增加了一点(多路)复用/(多路)分解服务而已。

### 3.3 无连接运输:UDP

在本节中,我们要仔细地研究一下UDP,看它是怎样工作的,能做些什么。我们鼓励你回过来看一下2.1节的内容,其中包括了UDP服务模型的概述,再看看2.7.1节,其中讨论了UDP上的套接字编程。

为了激发我们讨论UDP的热情,假如你对设计一个不提供不必要服务的最简化的运输层协议感兴趣。你将打算怎样做呢?你也许会首先考虑使用一个无所事事的运输层协议。特别是在发送方一侧,你可能会考虑将来自应用进程的数据直接交给网络层;在接收方一侧,你可能会考虑将从网络层到达的报文直接交给应用进程。而正如我们在前一节所学的,我们必须做一点点事,而不是什么都不做!运输层最低限度必须提供一种复用/分解服务,以便在网络层与正确的应用级进程之间传递数据。

由[RFC 768]定义的UDP只是做了运输协议能够做的最少工作。除了复用/分解功能及少量的差错检测外,它几乎没有对IP增加别的东西。实际上,如果应用程序开发人员选择UDP而不是TCP,则该应用程序差不多就是直接与IP打交道。UDP从应用进程得到数据,附加上用于多路复用/分解服务的源和目的端口号字段,以及两个其他的小字段,然后将形成的报文段交给网络层。网络层将该运输层报文段封装到一个IP数据报中,然后尽力而为地尝试将此报文段交付给接收主机。如果该报文段到达接收主机,UDP使用目的端口号将报文段中的数据交付给正确的应用进程。值得注意的是,使用UDP时,在发送报文段之前,发送方和接收方的运输层实体之间没有握手。正因为如此,UDP被称为是无连接的。

DNS是一个通常使用UDP的应用层协议的例子。当一台主机中的DNS应用程序想要



进行一次查询时，它构造了一个 DNS 查询报文并将其交给 UDP。无须执行任何与运行在目的端系统中的 UDP 实体之间的握手，主机端的 UDP 为此报文添加首部字段，然后将形成的报文段交给网络层。网络层将此 UDP 报文段封装进一个 IP 数据报中，然后将其发送给一个名字服务器。在查询主机中的 DNS 应用程序则等待对该查询的响应。如果它没有收到响应（可能是由于底层网络丢失了查询或响应），则要么试图向另一个名字服务器发送该查询，要么通知调用的应用程序它不能获得响应。

现在你也许想知道，为什么应用开发人员宁愿在 UDP 之上构建应用，而不选择在 TCP 上构建应用？既然 TCP 提供了可靠数据传输服务，而 UDP 不能提供，那么 TCP 是否总是首选的呢？答案是否定的，因为有许多应用更适合用 UDP，原因主要以下几点：

- 关于何时、发送什么数据的应用层控制更为精细。采用 UDP 时，只要应用进程将数据传递给 UDP，UDP 就会将此数据打包进 UDP 报文段并立即将其传递给网络层。在另一方面，TCP 有一个拥塞控制机制，以便当源和目的主机间的一条或多条链路变得极度拥塞时来遏制运输层 TCP 发送方。TCP 仍将继续重新发送数据报文段直到目的主机收到此报文并加以确认，而不管可靠交付需要用多长时间。因为实时应用通常要求最小的发送速率，不希望过分地延迟报文段的传送，且能容忍一些数据丢失，TCP 服务模型并不是特别适合这些应用的需要。如后面所讨论的，这些应用可以使用 UDP，并作为应用的一部分来实现所需的、超出 UDP 的不提供不必要的报文段交付服务之外的额外功能。
- 无需连接建立。如我们后面所讨论的，TCP 在开始数据传输之前要经过三次握手。UDP 却不需要任何准备即可进行数据传输。因此 UDP 不会引入建立连接的时延。这可能是 DNS 运行在 UDP 之上而不是运行在 TCP 之上的主要原因（如果运行在 TCP 上，则 DNS 会慢得多）。HTTP 使用 TCP 而不是 UDP，因为对于具有文本数据的 Web 网页来说，可靠性是至关重要的。但是，如我们在 2.2 节中简要讨论的那样，HTTP 中的 TCP 连接建立时延对于与下载 Web 文档相关的时延来说是一个重要因素。
- 无连接状态。TCP 需要在端系统中维护连接状态。此连接状态包括接收和发送缓存、拥塞控制参数以及序号与确认号的参数。我们将在 3.5 节看到，要实现 TCP 的可靠数据传输服务并提供拥塞控制，这些状态信息是必要的。在另一方面，UDP 不维护连接状态，也不跟踪这些参数。因此，某些专门用于某种特定应用的服务器当应用程序运行在 UDP 之上而不是运行在 TCP 上时，一般都能支持更多的活跃客户。
- 分组首部开销小。每个 TCP 报文段都有 20 字节的首部开销，而 UDP 仅有 8 字节的开销。

图 3-6 列出了流行的因特网应用及其所使用的运输协议。如我们所期望的那样，电子邮件、远程终端访问、Web 及文件传输都运行在 TCP 之上。因为所有这些应用都需要 TCP 的可靠数据传输服务。无论如何，有很多重要的应用是运行在 UDP 上而不是 TCP 上。UDP 被用于 RIP 路由选择表的更新（参见 4.6.1 节）。因为这些更新被周期性地发送（通常每 5 分钟一次），更新的丢失能被最近的更新所替代，因此丢包、过期的更新是无用的。UDP 也用于承载网络管理数据（SNMP，参见第 9 章）。在这种场合下，UDP 要优于 TCP，因为网络管理应用程序通常必须在该网络处于重压状态时运行，而正是在这个时候可靠



的、拥塞受控的数据传输难以实现。此外，如我们前面所述，DNS 运行在 UDP 之上，从而避免了 TCP 的连接创建时延。

| 应用      | 应用层协议  | 下面的运输协议   |
|---------|--------|-----------|
| 电子邮件    | SMTP   | TCP       |
| 远程终端访问  | Telnet | TCP       |
| Web     | HTTP   | TCP       |
| 文件传输    | FTP    | TCP       |
| 远程文件服务器 | NFS    | 通常 UDP    |
| 流式多媒体   | 通常专用   | UDP 或 TCP |
| 因特网电话   | 通常专用   | UDP 或 TCP |
| 网络管理    | SNMP   | 通常 UDP    |
| 路由选择协议  | RIP    | 通常 UDP    |
| 名字转换    | DNS    | 通常 UDP    |

图 3-6 流行的因特网应用及其下面的运输协议

如图 3-6 所示，UDP 和 TCP 现在都用于多媒体应用，如因特网电话、实时视频会议、流式存储音频与视频。我们将在第 7 章仔细学习这些应用。我们刚说过，既然所有这些应用都能容忍少量的分组丢失，因此可靠数据传输对于这些应用的成功并不是至关重要的。此外，TCP 的拥塞控制会导致如因特网电话、视频会议之类的实时应用性能变得很差。由于这些原因，多媒体应用开发人员通常将这些应用运行在 UDP 之上而不是 TCP 之上。然而，TCP 被越来越多地应用于流式多媒体传输中。例如，[Sripanidkulchai 2004] 发现大约 75% 的按需和实况流式多媒体使用了 TCP。当分组丢包率低时，并且为了安全原因，某些机构阻塞 UDP 流量（参见第 8 章），对于流式媒体传输来说，TCP 变得越来越有吸引力了。

虽然目前通常这样做，但在 UDP 之上运行多媒体应用是有争议的。如我们前面所述，UDP 没有拥塞控制。但是，需要拥塞控制来预防网络进入一种拥塞状态，此时只能做很少的有用工作。如果每个人都启动流式高比特率视频而不使用任何拥塞控制的话，就会使路由器中有大量的分组溢出，以至于几乎没有 UDP 分组能成功地通过源到目的的路径传输。况且，由无控制的 UDP 发送方引入的高丢包率将引起 TCP 发送方（如我们将看到的那样，TCP 遇到拥塞将减小它们的发送速率）大大地减小它们的速率。因此，UDP 中缺乏拥塞控制能够导致 UDP 发送方和接收方之间的高丢包率，并挤垮了 TCP 会话，这是一个潜在的严重问题 [Floyd 1999]。很多研究人员已提出了一些新机制，以促使所有的数据源（包括 UDP 源）执行自适应的拥塞控制 [Mahdavi 1997；Floyd 2000；Kohler 2006；RFC 4340]。

在讨论 UDP 报文段结构之前，我们要提一下，使用 UDP 的应用是可以实现可靠数据传输的。这可通过在应用程序自身中建立可靠性机制来完成（例如，可通过增加确认与重传机制来实现，如采用我们将在下一节学习的一些机制）。但这并不是无足轻重的任务，它会使应用开发人员长时间地忙于调试。无论如何，将可靠性直接构建于应用程序中可以使其“左右逢源”。也就是说应用进程可以进行可靠通信，而无需受制于由 TCP 拥塞控制机制而无需受制于传输速率限制。



3.3.1 UDP 报文段结构

UDP 报文段结构如图 3-7 所示，它由 RFC 768 定义。应用层数据占用 UDP 报文段的数据字段。例如，对于 DNS 应用，数据字段要么包含一个查询报文，要么包含一个响应报文。对于流式音频应用，音频抽样数据填充到数据字段。UDP 首部只有 4 个字段，每个字段由两个字节组成。如前一节所讨论的，通过端口号可以使目的主机将应用数据交给运行在目的端系统中的相应进程（即执行分解功能）。长度字段指示了在 UDP 报文段中的字节数（首部加数据）。因为数据字段的长度在一个 UDP 段中不同于在另一个段中，故需要一个明确的长度。接收方使用检验和来检查在该报文段中是否出现了差错。实际上，计算检验和时，除了 UDP 报文段以外还包括了 IP 首部的一些字段。但是我们忽略这些细节，以便能从整体上看问题。下面我们将讨论检验和的计算。在 5.2 节中将描述差错检测的基本原理。长度字段指明了包括首部在内的 UDP 报文段长度（以字节为单位）。

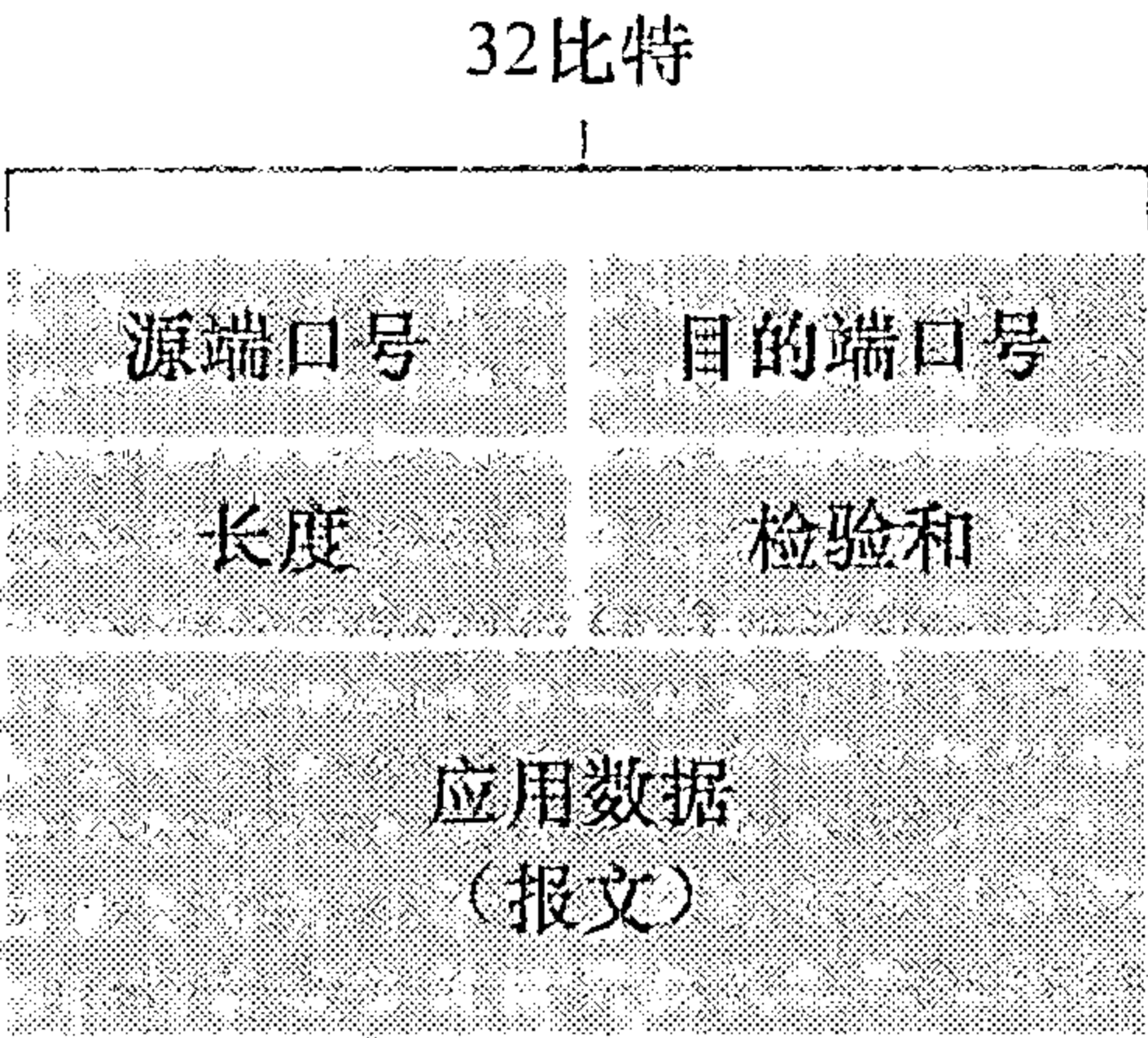


图 3-7 UDP 报文段结构

3.3.2 UDP 检验和

UDP 检验和提供了差错检测功能。这就是说，检验和用于确定当 UDP 报文段从源到达目的地移动时，其中的比特是否发生了改变（例如，由于链路中的噪声干扰或者存储在路由器中时引入问题）。发送方的 UDP 对报文段中的所有 16 比特字的和进行反码运算，求和时遇到的任何溢出都被回卷。得到的结果被放在 UDP 报文段中的检验和字段。下面给出一个计算检验和的简单例子。在 RFC 1071 中可以找到有效实现的细节，还可在 [Stone 1998；Stone 2000] 中找到它处理真实数据的性能。举例来说，假定我们有下面 3 个 16 比特的字：

0110011001100000  
0101010101010101  
1000111100001100

这些 16 比特字的前两个之和是：

0110011001100000  
0101010101010101  
-----  
1011101110110101

再将上面的和与第三个字相加，得出：

1011101110110101  
1000111100001100  
-----  
0100101011000010

注意到最后一次加法有溢出，它要被回卷。反码运算就是将所有的 0 换成 1，所有的 1 转换成 0。因此，该和 0100101011000010 的反码运算结果是 1011010100111101，这变为了检验和。在接收方，全部的 4 个 16 比特字（包括检验和）加在一起。如果该分组中没有引入差错，则显然在接收方处该和将是 1111111111111111。如果这些比特之一是 0，那



么我们就知道该分组中已经出现了差错。

你可能想知道为什么 UDP 首先提供了检验和，就像许多链路层协议（包括流行的以太网协议）也提供了差错检测那样。其原因是由于不能保证源和目的之间的所有链路都提供差错检测；这就是说，也许这些链路中的一条可能使用没有差错检测的协议。此外，即使报文段经链路正确地传输，当报文段存储在某个路由器的内存中时，也可能引入比特差错。在既无法确保逐链路的可靠性，又无法确保内存中的差错检测的情况下，如果端到端数据传输服务要提供差错检测，UDP 就必须在端到端基础上在运输层提供差错检测。这是一个在系统设计中被称颂的**端到端原则**（end-end principle）的例子 [Saltzer 1984]，该原则表述为因为某种功能（在此时为差错检测）必须基于端到端实现：“与在较高级别提供这些功能的代价相比，在较低级别上设置的功能可能是冗余的或几乎没有价值的。”

因为假定 IP 是可以运行在任何第二层协议之上的，运输层提供差错检测作为一种保险措施是非常有用的。虽然 UDP 提供差错检测，但它对差错恢复无能为力。UDP 的某种实现只是丢弃受损的报文段；其他实现是将受损的报文段交给应用程序并给出警告。

至此结束了关于 UDP 的讨论。我们将很快看到 TCP 为应用提供了可靠数据传输及 UDP 所不能提供的其他服务。TCP 自然要比 UDP 复杂得多。然而，在讨论 TCP 之前，我们后退一步，先来讨论一下可靠数据传输的基本原理是有用的。

### 3.4 可靠数据传输原理

在本节中，我们在通常情况下考虑可靠数据传输的问题。因为可靠数据传输的实现问题不仅在运输层出现，也会在链路层以及应用层出现，这时讨论它是恰当的。因此，一般性问题对网络来说更为重要。如果的确要将所有网络中最为重要的“前 10 个”问题排名的话，可靠数据传输将是名列榜首的候选者。在下一节中，我们将学习 TCP，尤其要说明 TCP 所采用的许多原理，而这些正是我们打算描述的内容。

图 3-8 图示说明了我们学习可靠数据传输的框架。为上层实体提供的服务抽象是：数据可以通过一条可靠的信道进行传输。借助于可靠信道，传输数据比特就不会受到损坏（由 0 变为 1，或者相反）或丢失，而且所有数据都是按照其发送顺序进行交付。这恰好就是 TCP 向调用它的因特网应用所提供的服务模型。

实现这种服务抽象是**可靠数据传输协议**（reliable data transfer protocol）的责任。由于可靠数据传输协议的下层协议也许是不可靠的，因此这是一项困难的任务。例如，TCP 是在不可靠的（IP）端到端网络层之上实现的可靠数据传输协议。更一般的情况是，两个可靠通信端点的下层可能是由一条物理链路（如在链路级数据传输协议的场合下）组成或是由一个全球互联网络（如在运输级协议的场合下）组成。然而，就我们的目的而言，我们可将较低层直接视为不可靠的点对点信道。

在本节中，考虑到底层信道模型越来越复杂，我们将不断地开发一个可靠数据传输协议的发送方和接收方。例如，我们将考虑当底层信道能够损坏比特或丢失整个分组时，需要什么样的协议机制。这里贯穿我们讨论始终的一个假设是分组将以它们发送的次序进行交付，某些分组可能会丢失；这就是说，底层信道将不会对分组重排序。图 3-8b 图示说明了用于数据传输协议的接口。通过调用 `rdt_send()` 函数，可以调用数据传输协议的发送



方。它将要发送的数据交付给位于接收方的较高层。（这里 rdt 表示可靠数据传输协议，\_send 指示 rdt 的发送端正在被调用。开发任何协议的第一步就是要选择一个好的名字！）在接收端，当分组从信道的接收端到达时，将调用 rdt\_rcv()。当 rdt 协议想要向较高层交付数据时，将通过调用 deliver\_data() 来完成。后面，我们将使用术语“分组”而不用运输层的“报文段”。因为本节研讨的理论适用于一般的计算机网络，而不只是用于因特网运输层，所以这时采用通用术语“分组”也许更为合适。

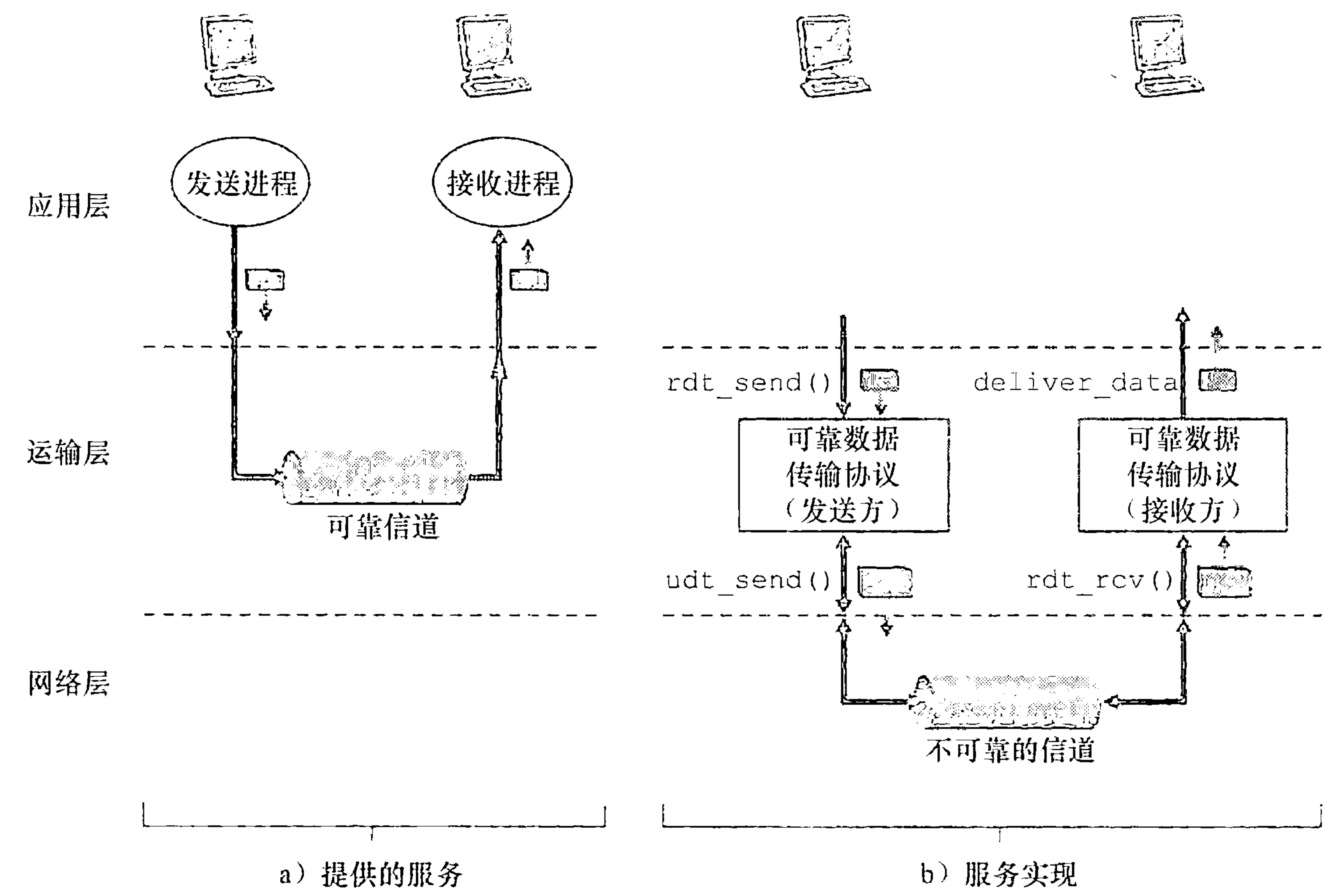


图 3-8 可靠数据传输：服务模型与服务实现

在本节中，我们仅考虑单向数据传输（unidirectional data transfer）的情况，即数据传输是从发送端到接收端的。可靠的双向数据传输（bidirectional data transfer）（即全双工数据传输）情况从概念上讲不会更难，但解释起来更为单调乏味。虽然我们只考虑单向数据传输，注意到下列事实是重要的，我们的协议也需要在发送端和接收端两个方向上传输分组，如图 3-8 所示。我们很快会看到，除了交换含有待传送的数据的分组之外，rdt 的发送端和接收端还需往返交换控制分组。rdt 的发送端和接收端都要通过调用 udt\_send() 发送分组给对方（其中 udt 表示不可靠数据传输）。

3.4.1 构造可靠数据传输协议

我们现在一步步地研究一系列协议，它们一个比一个更为复杂，最后得到一个无错、可靠的数据传输协议。

1. 经完全可靠信道的可靠数据传输：rdt 1.0

首先，我们考虑最简单的情况，即底层信道是完全可靠的。我们称该协议为 rdt1.0，该协议本身是简单的。图 3-9 显示了 rdt 1.0 发送方和接收方的有限状态机（Finite-State



Machine, FSM) 的定义。图 3-9a 中的 FSM 定义了发送方的操作, 图 3-9b 中的 FSM 定义了接收方的操作。注意到下列问题是重要的, 发送方和接收方有各自的 FSM。图 3-9 中发送方和接收方的 FSM 每个都只有一个状态。FSM 描述图中的箭头指示了协议从一个状态变迁到另一个状态。(因为图 3-9 中的每个 FSM 都只有一个状态, 因此变迁必定是从一个状态返回到自身; 我们很快将看到更复杂的状态图。) 引起变迁的事件显示在表示变迁的横线上方, 事件发生时所采取的动作显示在横线下方。如果对一个事件没有动作, 或没有就事件发生而采取了一个动作, 我们将在横线上方或下方使用符号  $\Lambda$ , 以分别明确地表示缺少动作或事件。FSM 的初始状态用虚线表示。尽管图 3-9 中的 FSM 只有一个状态, 但马上我们就将看到多状态的 FSM, 因此标识每个 FSM 的初始状态是非常重要的。

rdt 的发送端只通过 `rdt_send(data)` 事件接受来自较高层的数据, 产生一个包含该数据的分组 (经由 `make_pkt(data)` 动作), 并将分组发送到信道中。实际上, `rdt_send(data)` 事件是由较高层应用的过程调用产生的 (例如, `rdt_send()`)。

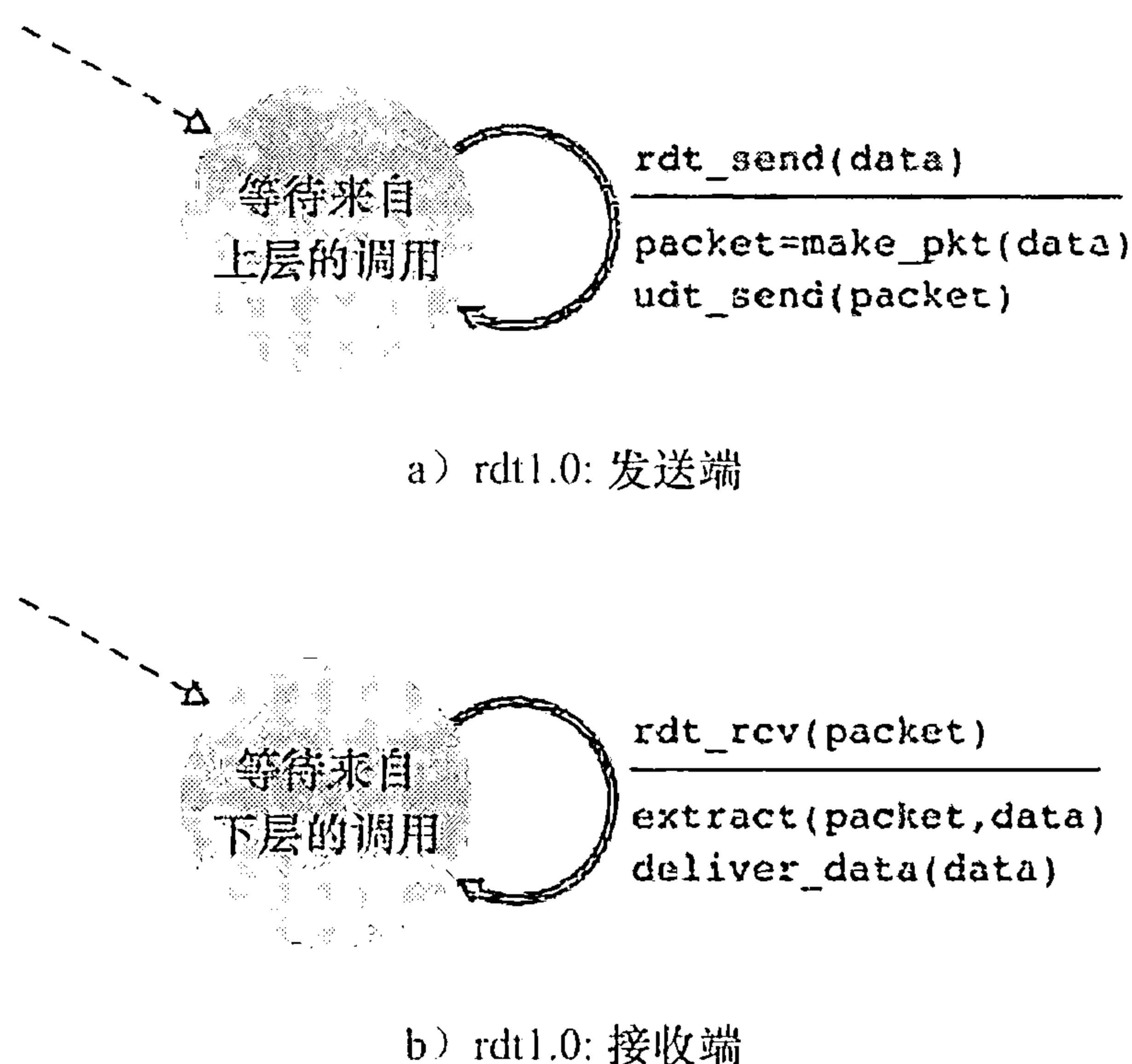


图 3-9 rdt1.0: 用于完全可靠信道的协议

在接收端, rdt 通过 `rdt_rcv(packet)` 事件从底层信道接收一个分组, 从分组中取出数据 (经由 `extract(packet, data)` 动作), 并将数据上传给较高层 (通过 `deliver_data(data)` 动作)。实际上, `rdt_rcv(packet)` 事件是由较低层协议的过程调用产生的 (例如, `rdt_rcv()`)。

在这个简单的协议中, 一个单元数据与一个分组没差别。而且, 所有分组是从发送方流向接收方; 有了完全可靠的信道, 接收端就不需要提供任何反馈信息给发送方, 因为不必担心出现差错! 注意到我们也已经假定了接收方接收数据的速率能够与发送方发送数据的速率一样快。因此, 接收方没有必要请求发送方慢一点!

## 2. 经具有比特差错信道的可靠数据传输: rdt 2.0

底层信道更为实际的模型是分组中的比特可能受损。在分组的传输、传播或缓存的过程中, 这种比特差错通常会出现在网络的物理部件中。我们眼下还将继续假定所有发送的分组 (虽然有些比特可能受损) 将按其发送的顺序被接收。

在研发一种经这种信道进行可靠通信的协议之前, 首先考虑一下人们会怎样处理这类情形。考虑一下你自己是怎样通过电话口述一条长消息的。在通常情况下, 报文接收者在听到、理解并记下每句话后可能会说 “OK”。如果报文接收者听到一句含糊不清的话时, 他可能要求你重复刚才那句话。这种口述报文协议使用了肯定确认 (positive acknowledgment) (“OK”) 与否定确认 (negative acknowledgment) (“请重复一遍”)。这些控制报文使得接收方可以让发送方知道哪些内容被正确接收, 哪些内容接收有误并因此需要重复。在计算机网络环境中, 基于这样重传机制的可靠数据传输协议称为自动重传请求 (Automatic Repeat reQuest, ARQ) 协议。

基本上, ARQ 协议中还需要另外三种协议功能来处理存在比特差错的情况:



- 差错检测。首先，需要一种机制以使接收方检测到何时出现了比特差错。前一节讲到，UDP 使用因特网检验和字段正是为了这个目的。在第 5 章中，我们将更详细地学习差错检测和纠错技术。这些技术使接收方可以检测并可能纠正分组中的比特差错。此刻，我们只需知道这些技术要求有额外的比特（除了待发送的初始数据比特之外的比特）从发送方发送到接收方；这些比特将被汇集在 rdt 2.0 数据分组的分组检验和字段中。
- 接收方反馈。因为发送方和接收方通常在不同端系统上执行，可能相隔数千英里，发送方要了解接收方情况（此时为分组是否被正确接收）的唯一途径就是让接收方提供明确的反馈信息给发送方。在口述报文情况下回答的“肯定确认”（ACK）和“否定确认”（NAK）就是这种反馈的例子。类似地，我们的 rdt 2.0 协议将从接收方向发送方回送 ACK 与 NAK 分组。理论上，这些分组只需要一个比特长；如用 0 表示 NAK，用 1 表示 ACK。
- 重传。接收方收到有差错的分组时，发送方将重传该分组文。

图 3-10 说明了表示 rdt 2.0 的 FSM，该数据传输协议采用了差错检测、肯定确认与否定确认。

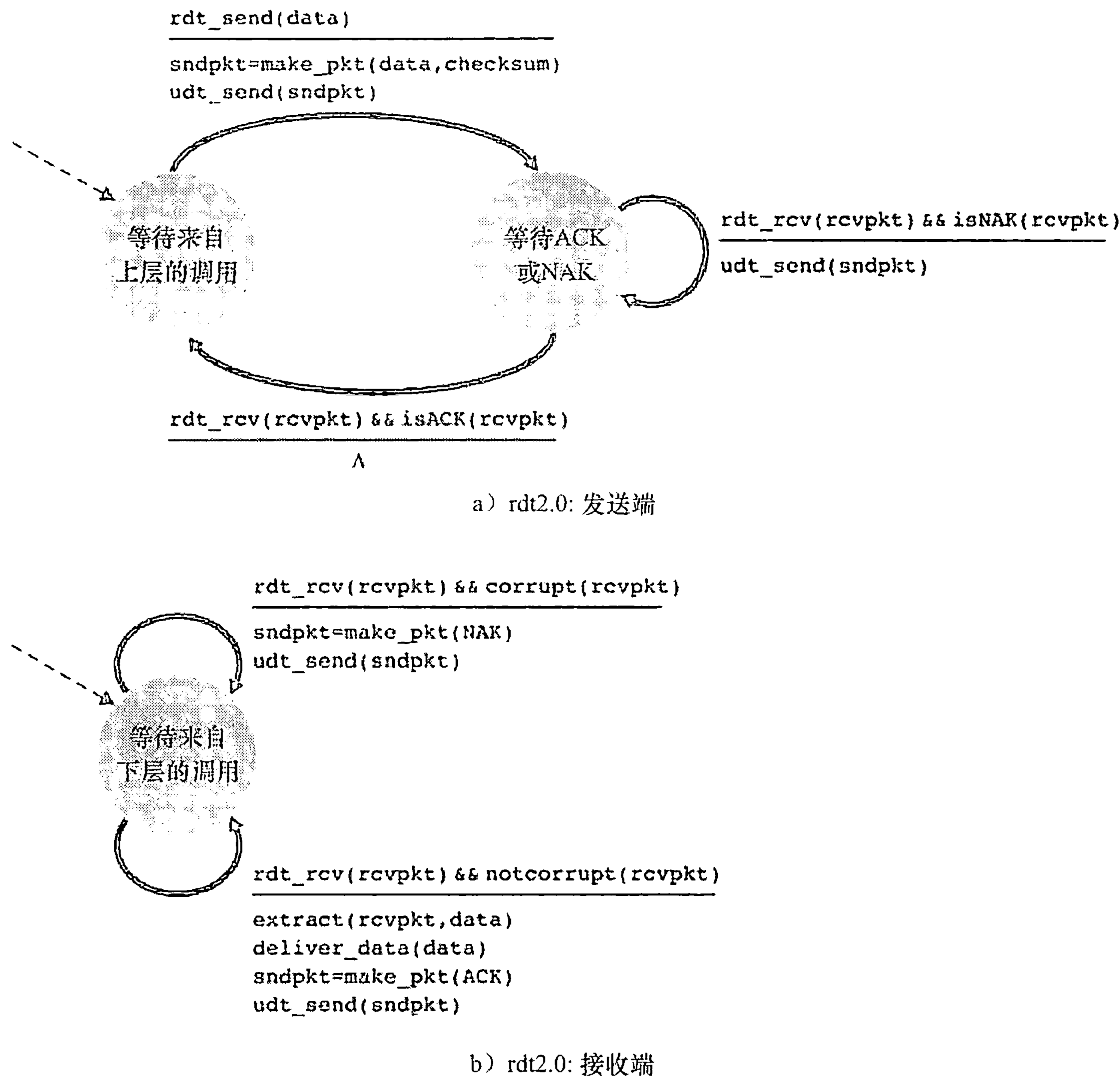


图 3-10 rdt 2.0：用于具有比特差错信道的协议



rdt 2.0 的发送端有两个状态。在最左边的状态中，发送端协议正等待来自上层传下来的数据。当产生 `rdt_send(data)` 事件时，发送方将产生一个包含待发送数据的分组 (`sndpkt`)，带有检验和（例如，就像在 3.3.2 节讨论的对 UDP 报文段使用的方法），然后经由 `udt_send(sndpkt)` 操作发送该分组。在最右边的状态中，发送方协议等待来自接收方的 ACK 或 NAK 分组。如果收到一个 ACK 分组（图 3-10 中符号 `rdt_rev(rcvpkt) && isACK(rcvpkt)` 对应的事件），则发送方知道最近发送的分组已被正确接收，因此协议返回到等待来自上层的数据的状态。如果收到一个 NAK 分组，该协议重传最后一个分组并等待接收方为响应重传分组而回送的 ACK 和 NAK。注意到下列事实很重要：当发送方处于等待 ACK 或 NAK 的状态时，它不能从上层获得更多的数据；这就是说，`rdt_send()` 事件不可能出现；仅当接收到 ACK 并离开该状态时才能发生这样的事件。因此，发送方将不会发送一块新数据，除非发送方确信接收方已正确接收当前分组。由于这种行为，rdt 2.0 这样的协议被称为停等（stop-and-wait）协议。

rdt 2.0 接收方的 FSM 仍然只有一个状态。当分组到达时，接收方要么回答一个 ACK，要么回答一个 NAK，这取决于收到的分组是否受损。在图 3-10 中，符号 `rdt_rev(rcvpkt) && corrupt(rcvpkt)` 对应于收到一个分组并发现有错的事件。

rdt 2.0 协议看起来似乎可以运行了，但遗憾的是，它存在一个致命的缺陷。尤其是我们没有考虑到 ACK 或 NAK 分组受损的可能性！（在继续研究之前，你应该考虑怎样解决这个问题。）遗憾的是，我们细小的疏忽并非像它看起来那么无关紧要。至少，我们需要在 ACK/NAK 分组中添加检验和比特以检测这样的差错。更难的问题是协议应该怎样纠正 ACK 或 NAK 分组中的差错。这里的难点在于，如果一个 ACK 或 NAK 分组受损，发送方无法知道接收方是否正确接收了上一块发送的数据。

考虑处理受损 ACK 和 NAK 时的 3 种可能性：

- 对于第一种可能性，考虑在口述报文情况下人可能的做法。如果说话者不理解来自接收方回答的“OK”或“请重复一遍”，说话者将可能问“你说什么？”（因此在我们的协议中引入了一种新型发送方到接收方的分组）。接收方则将复述其回答。但是如果说话者的“你说什么？”产生了差错，情况又会怎样呢？接收者不明白那句混淆的话是口述内容的一部分还是一个要求重复上次回答的请求，很可能回一句“你说什么？”。于是，该回答可能含糊不清了。显然，我们走上了一条困难重重之路。
- 第二种可能性是增加足够的检验和比特，使发送方不仅可以检测差错，还可恢复差错。对于会产生差错但不丢失分组的信道，这就可以直接解决问题。
- 第三种方法是，当发送方收到含糊不清的 ACK 或 NAK 分组时，只需重传当前数据分组即可。然而，这种方法在发送方到接收方的信道中引入了冗余分组（duplicate packet）。冗余分组的根本困难在于接收方不知道它上次所发送的 ACK 或 NAK 是否被发送方正确地收到。因此它无法事先知道接收到的分组是新的还是一次重传！

解决这个新问题的一个简单方法（几乎所有现有的数据传输协议中，包括 TCP，都采用了这种方法）是在数据分组中添加一新字段，让发送方对其数据分组编号，即将发送数据分组的序号（sequence number）放在该字段。于是，接收方只需要检查序号即可确定收